embarca tech | Residência Tecnológica em Sistemas Embarcados

Clock e Temporizadores

Unidade 4 | Capítulo 5 – Aula Síncrona (29/01/2025)

Prof. Dr Ricardo Menezes Prates

Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

- Introdução
  - Planejamento da aula
- *Clock* e Temporizadores
  - *Clock* do microcontrolador RP2040
  - Temporizadores
- *Exemplos de aplicação*
  - *Configuração de Clock e Temporização de Sistema*
  - Temporizador Repetitivo
  - Temporizador de um disparo (*One Shot*)
- Conclusões

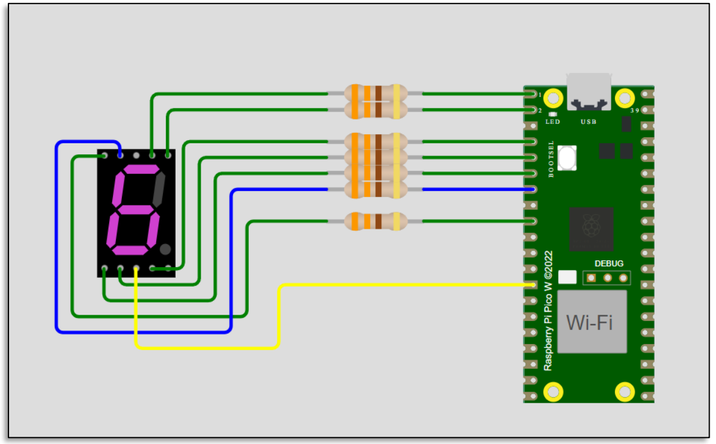
Introdução

Objetivos

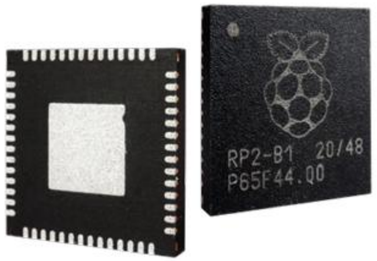
- Apresentar ao discente os conceitos básicos do ***clock*** e dos **temporizadores** presentes no microcontrolador RP2040 e na placa de desenvolvimento Raspberry Pi Pico W.

- Implementar diferentes exemplos de *temporização de hardware*, abrangendo o tipo repetitiva e de apenas um disparo (*One Shot*).

Código fonte



Planejamento da Aula



- Pré-requisitos para esta aula:

✓ C Básico - Implementação para o RP2040;

✓ Portas de Entrada e Saída (GPIO);

✓ Rotinas de *delay* básico;

✓ Rotinas de interrupção;

✓ Eletrônica básica – LEDs e resistores;

WOKWI

raspberrypi/**pico-sdk**

Planejamento da Aula

- O que ainda está por vir:

✓ Interfaces de comunicação serial;

✓ Modulação por Largura de Pulso - PWM;

✓ Conversor analógico digital;

"Se eu pude ver mais longe, foi porque me apoiei em ombros de gigantes" - frase popularmente atribuída a Sir Isaac Newton.

Clock e Temporizadores

Clock e Temporizadores

- Características do *clock*:

✓ Em sistemas digitais, todos os componentes internos tendem as ser sincronizados com uma forma de onda de temporização de referência denominada *clock*.

✓ Produzem sinais regulares e periódicos para sincronizar operações dentro de um sistema, e permitem a coordenação entre diferentes componentes.

✓ O *clock (relógio)* define a velocidade com que o microcontrolador executa as instruções.

Clock e Temporizadores

- Características do *clock*:

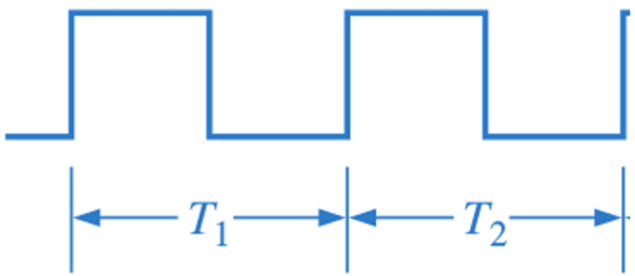
✓ É um sinal elétrico e periódico - cada intervalo entre os pulsos (período) é igual.

✓ O período do *clock* é o tempo entre a borda duas bordas de subida ou de descida.

✓ A **frequência** (f) é a taxa com que ela se repete por segundo e é medida em hertz (Hz).

✓ Quanto maior a frequência, maior a velocidade de processamento. Contudo, esta elevação pode resultar em maior consumo de energia.

T - Período do clock (s):



Frequência do clock (Hz):

$$f = \frac{1}{T}$$

Clock e Temporizadores

- Características do *clock*:

✓ O RP2040 possui uma arquitetura de *clock* bastante flexível e complexa – possui ajuste para múltiplos *clocks*.

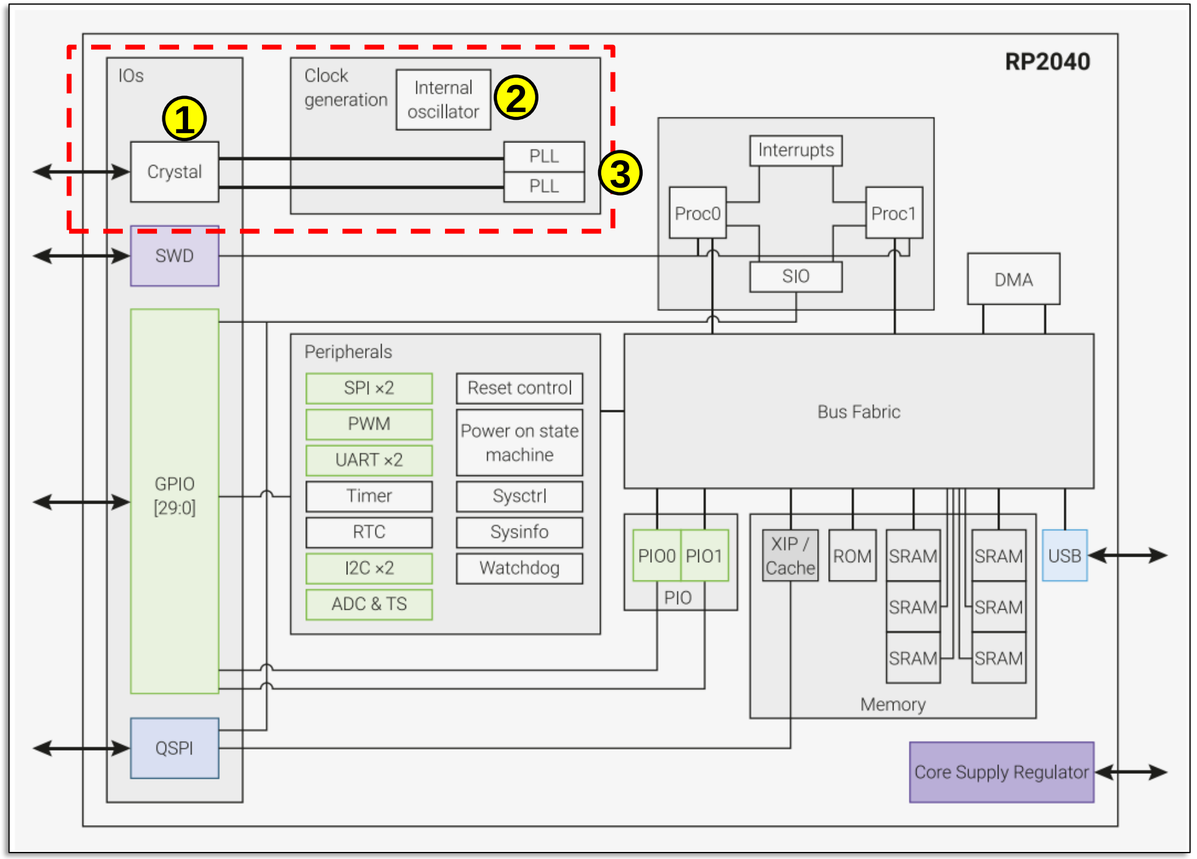
Este microcontrolador apresenta:

1) Oscilador externo - *Crystal Oscillator* (XOSC). ①

2) Oscilador interno - *Ring Oscillator* (ROSC). ②

3) 2 PLL (*Phase-Locked Loop*) – elevadores de frequência. ③

Arquitetura interna do microcontrolador rp2040.



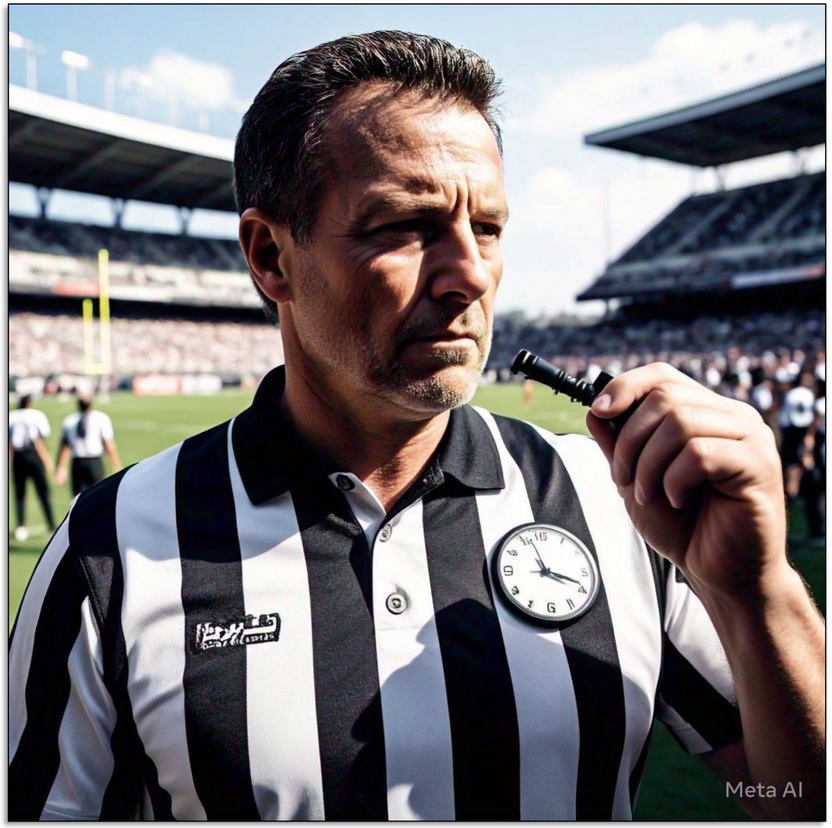
Clock e Temporizadores

- Características dos temporizadores (*timers*):

✓ Os temporizadores são **periféricos de microcontroladores** que são responsáveis por contagem de tempos ou eventos.

✓ Permitem a execução de tarefas em **intervalos específicos** sem a necessidade de intervenção constante do processador.

✓ Podem ser usados para controlar eventos periódicos de forma bastante eficiente.

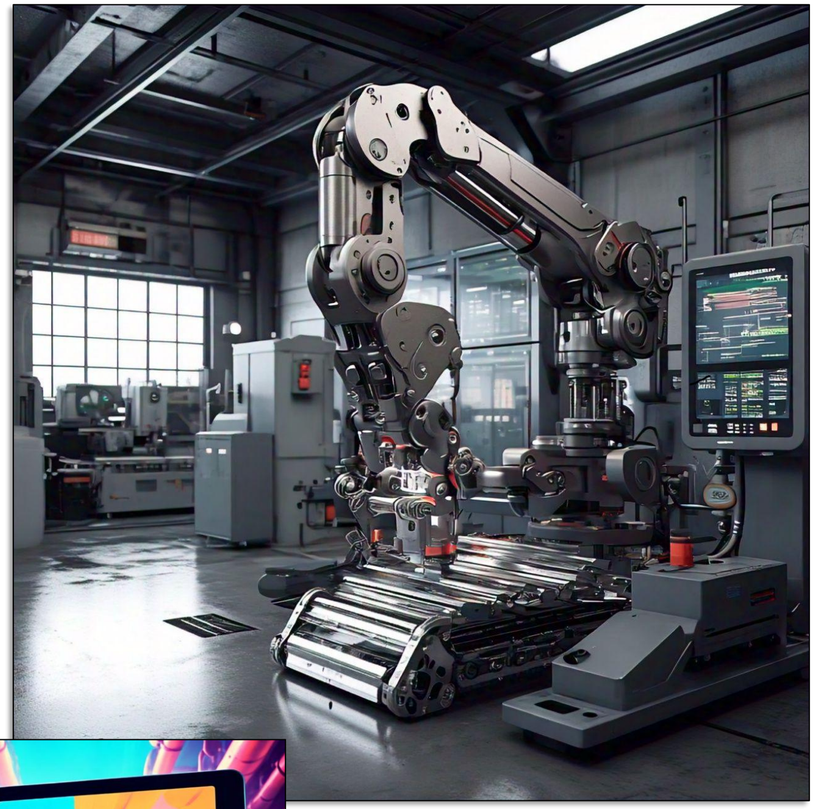


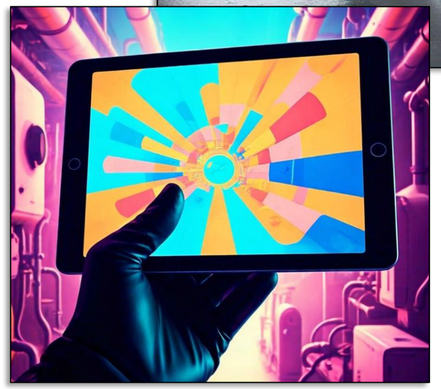
Clock e Temporizadores

- Tipos de temporizadores:

✓ **Temporizador periódico**: Disparam em intervalos regulares. Esses *Timers* podem ser utilizados para controlar temporizadores precisos em aplicações diversas.

✓ **Temporizador de um disparo (*One Shot*)**: É um recurso usado para executar uma ação após um certo período de tempo ter decorrido e, em seguida, parar.





Clock e Temporizadores

- Exemplos de aplicação de temporizadores:

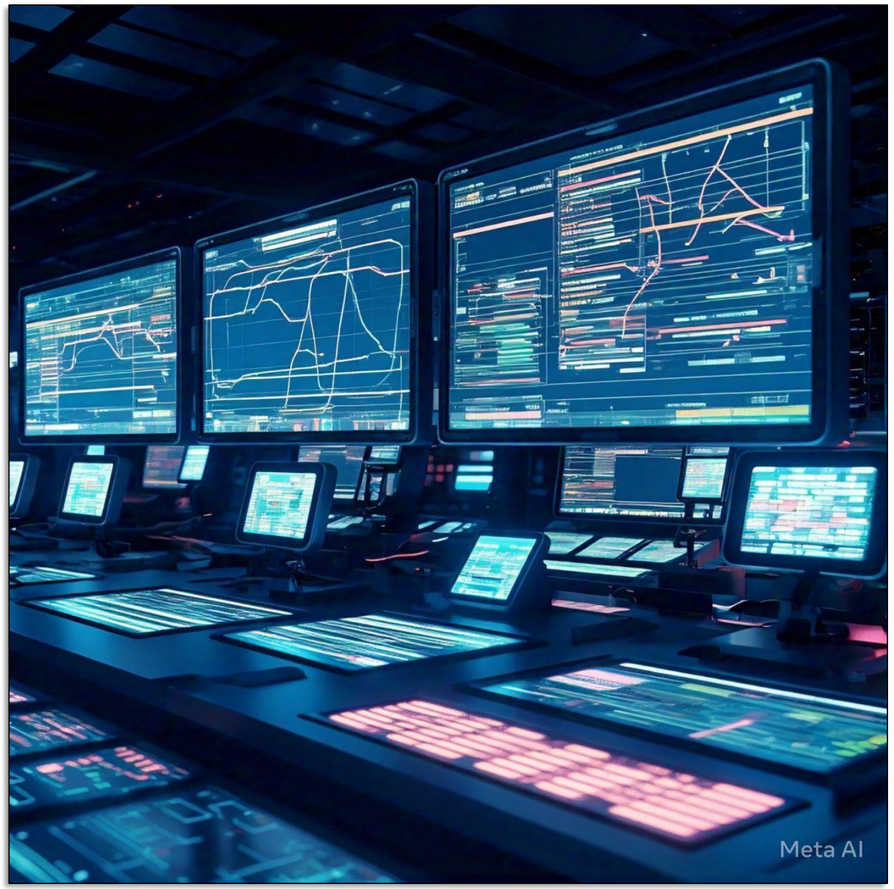
  - **Medição de tempo**: medir a duração de eventos, intervalos de tempo ou frequências.

  - **Controle de tempo**: Gerar atrasos (*delays*) precisos, controlar a temporização de processos e gerar interrupções.

  - **Geração de sinais**: Gerar sinais PWM para controle de motores, LEDs, etc.

  - **Sistemas de tempo real**: Implementar tarefas periódicas em sistemas embarcados.

  - **Implementação de protocolos de comunicação**: Controlar a temporização de sinais em protocolos como UART, SPI e I2C.



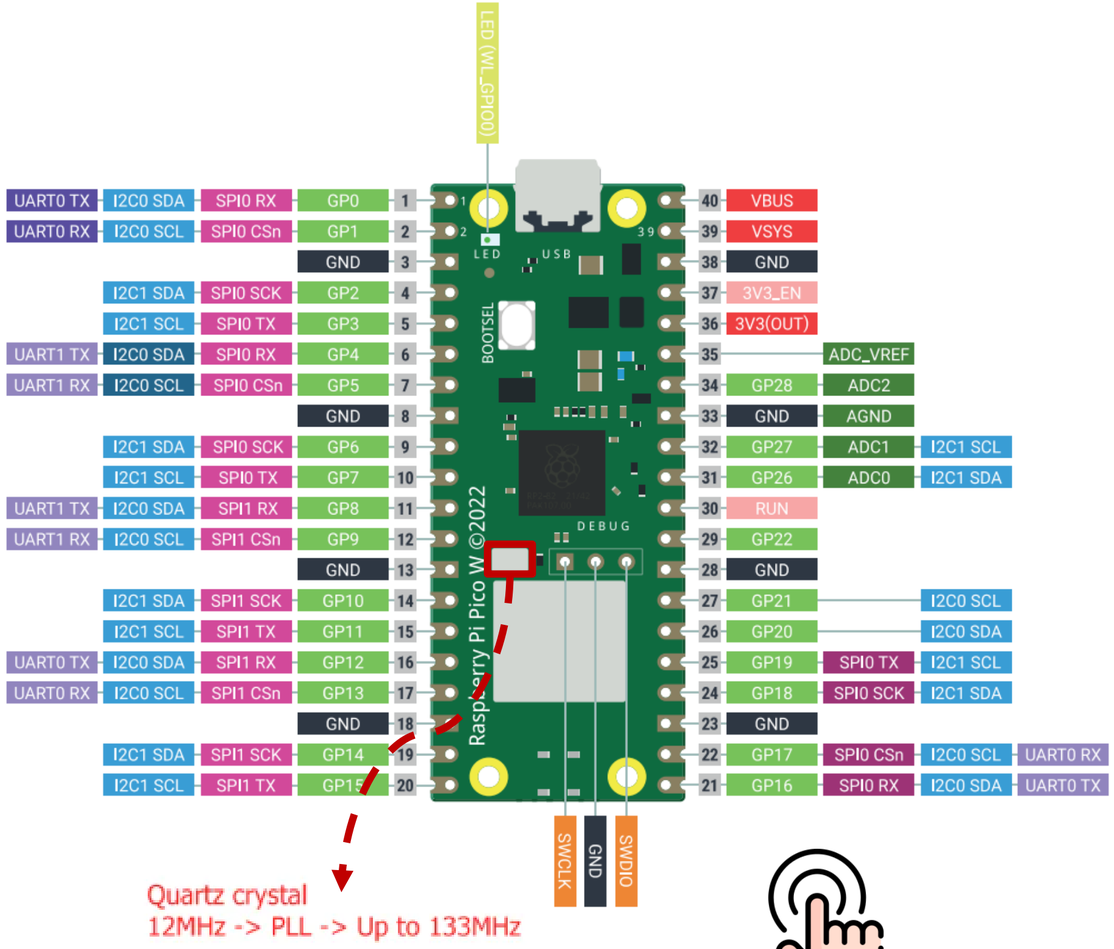
Clock do Microcontrolador RP2040

Clock - RP2040

- *Clocks* :

- No Raspberry Pi Pico W, o *clock* principal é gerado por um oscilador de cristal de 12 MHz.

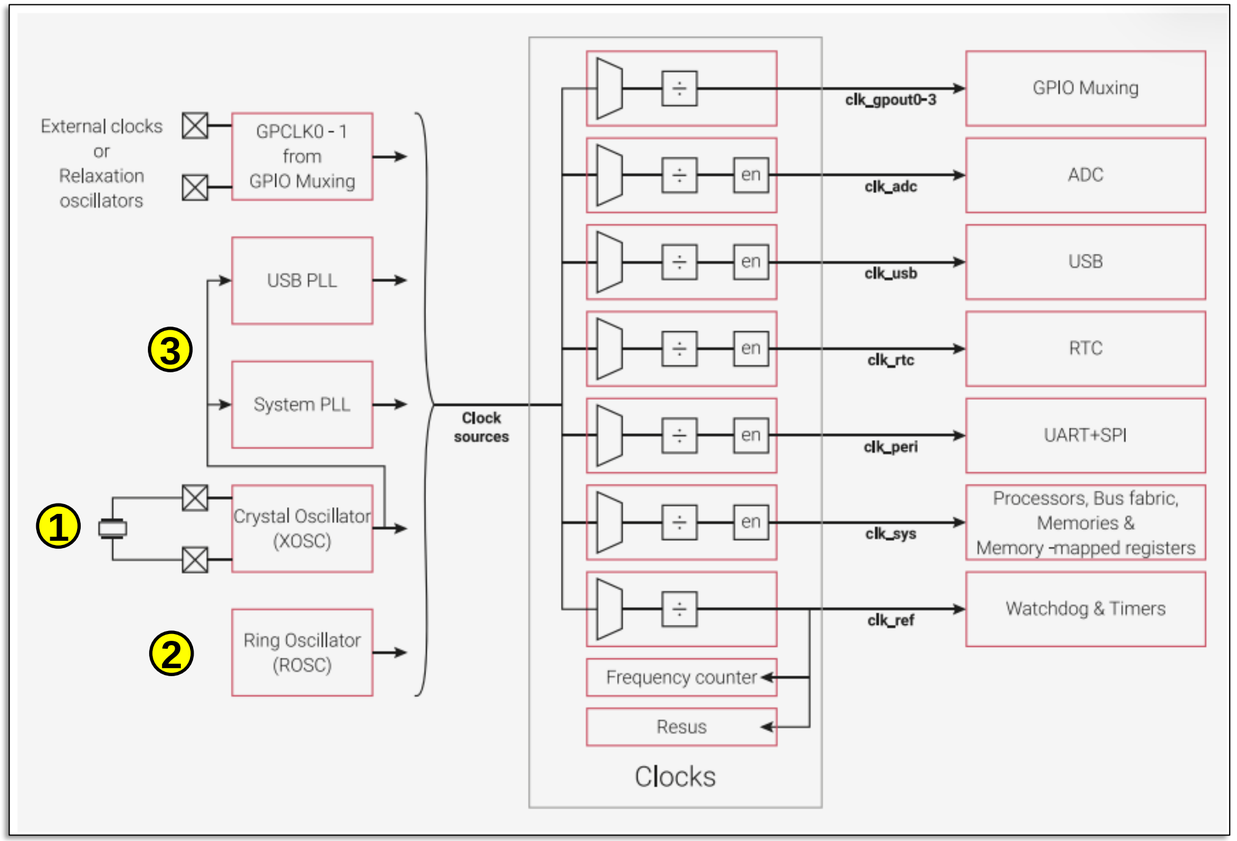
➢ Ele é multiplicado internamente para gerar *clocks* mais rápidos, como 125 MHz ou até mais, dependendo da aplicação.



Clock - RP2040

- *Clocks* :

✓ Visão geral.



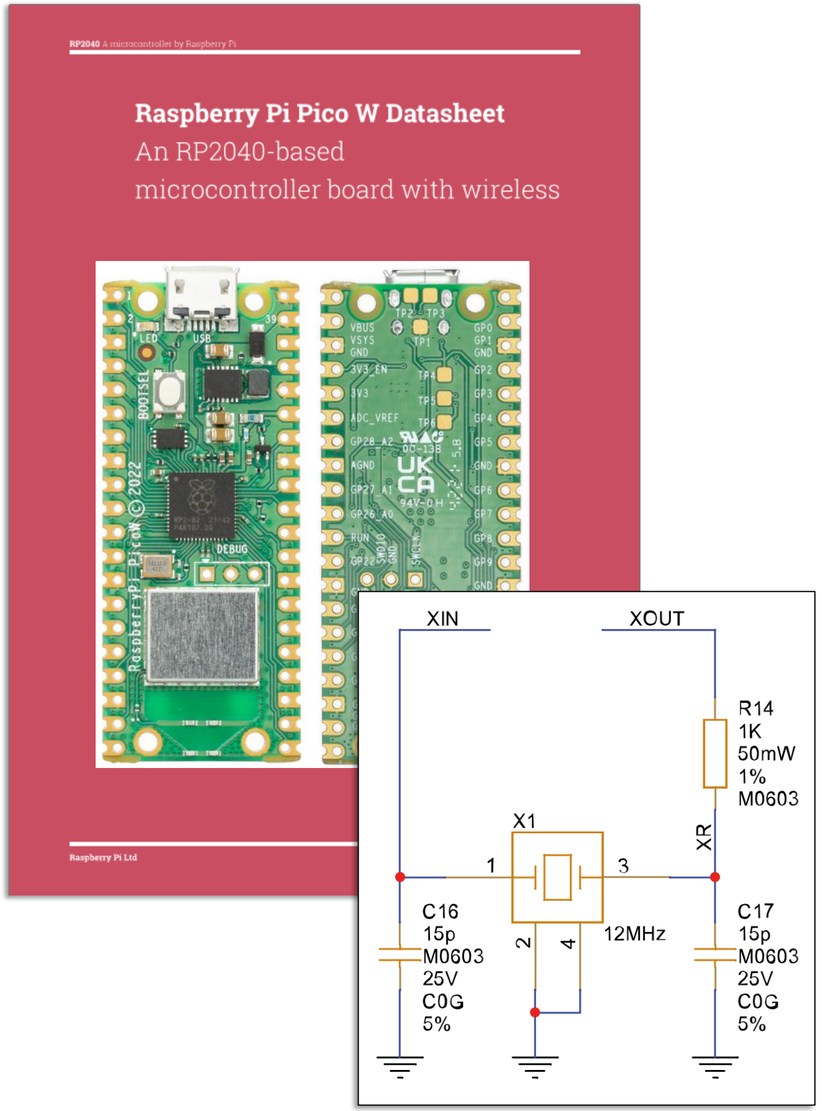
Exemplo.

```
imprimindo frequências:
pll_sys  = 100000kHz
pll_usb  = 48000kHz
rosc     = 5341kHz
clk_sys  = 100000kHz
clk_peri = 48000kHz
clk_usb  = 48000kHz
clk_adc  = 48000kHz
clk_rtc  = 47kHz
```

Clock - RP2040

1. *Crystal Oscillator* (XOSC):

- ✓ O oscilador de cristal (XOSC) utiliza um cristal externo para produzir um *clock* de referência preciso - **12MHz**.
- ✓ O RP2040 suporta de 1MHz a 15MHz.
- ✓ É distribuído para os **PLLs** (*Phase-Locked Loop*), que multiplicam a frequência XOSC para fornecer relógios precisos de alta velocidade Ex: 125MHz.



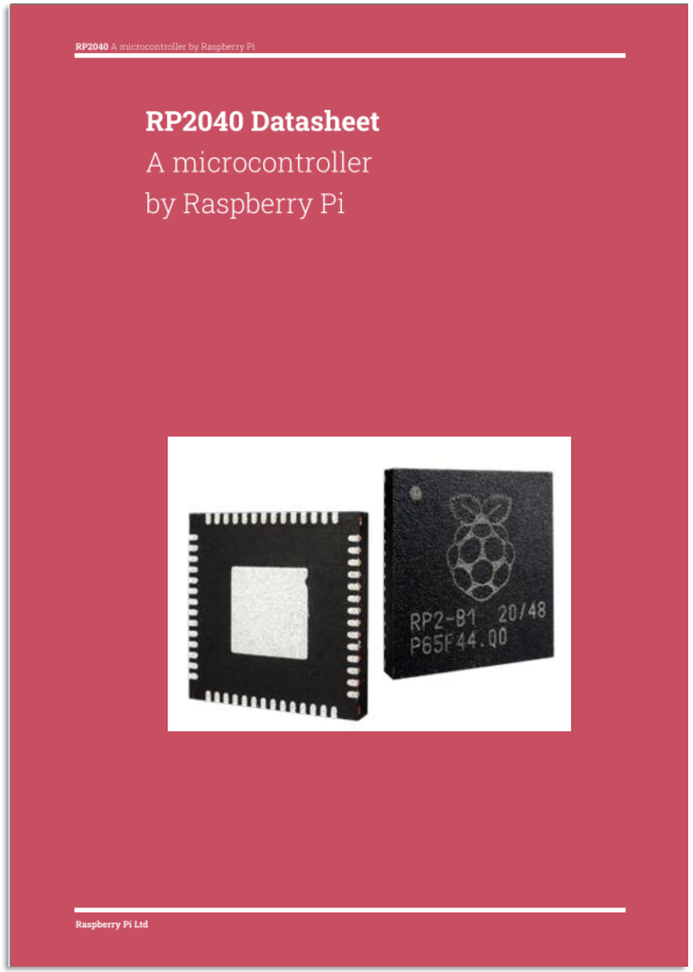
Clock - RP2040

1. *Crystal Oscillator* (XOSC) - 2.16. :

- ✓ O oscilador de cristal (XOSC) utiliza um cristal externo para produzir um *clock* de referência preciso - **12MHz**.

- ✓ O RP2040 suporta de 1MHz a 15MHz.

- ✓ É distribuído para os **PLLs** (*Phase-Locked Loop*), que multiplicam a frequência XOSC para fornecer relógios precisos de alta velocidade Ex: 125MHz.

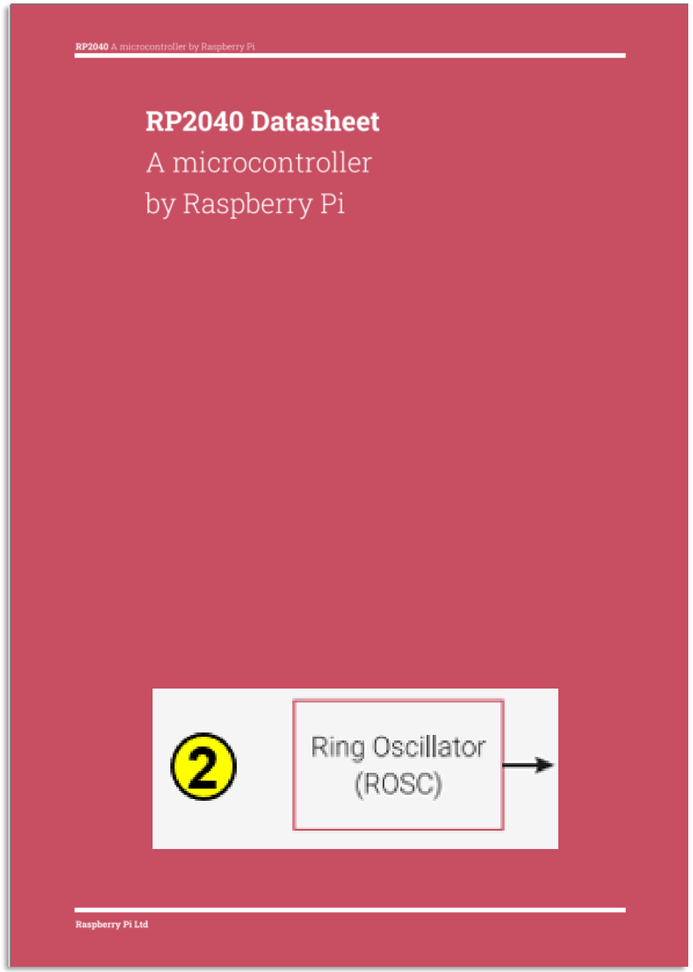


Clock - RP2040

2. Ring Oscillator (ROSC) - 2.17. :

- ✓ O oscilador em anel (ROSC) é um oscilador *on-chip* construído a partir de um anel de inversores.

- ✓ A frequência do ROSC é programável e pode fornecer diretamente um relógio de alta velocidade aos núcleos.



Clock - RP2040

2. Ring Oscillator (ROSC) - 2.17. :

- ✓ A sua frequência varia com o processo, a tensão e a temperatura (PVT).
- ✓ Pode não fornecer *clocks* adequados para componentes que exigem uma frequência precisa, como o RTC, USB e ADC.
- ✓ Durante o boot, o ROSC funciona a uma frequência nominal de 6,5MHz e é garantido que se situa no intervalo de 1,8MHz a 12MHz.

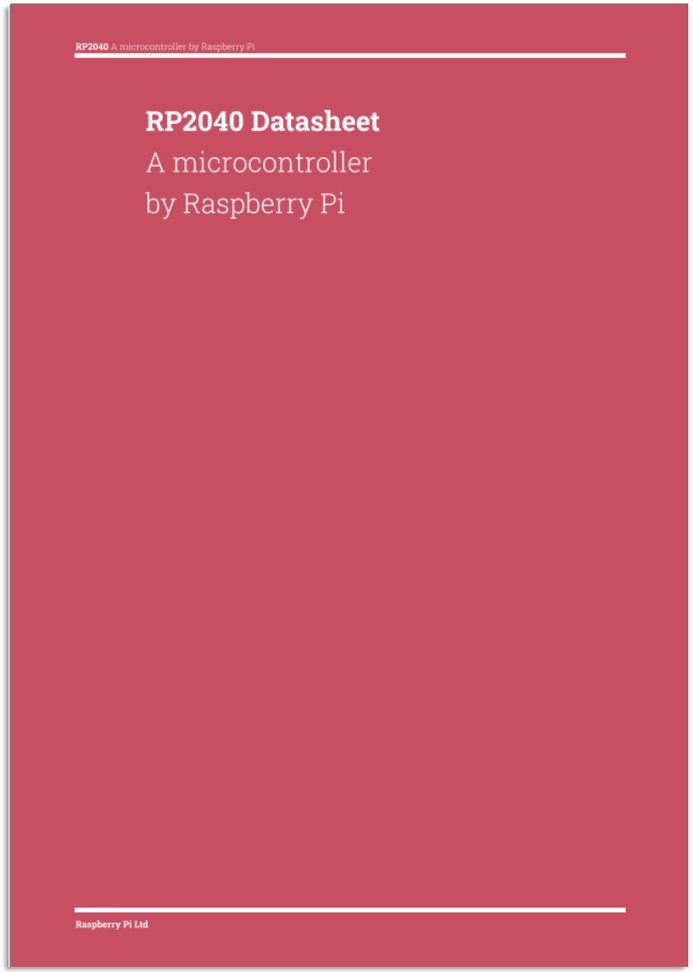
```
-- Intervalo de 5s
rosc     = 5325kHz
rosc     = 5328kHz
rosc     = 5329kHz
rosc     = 5329kHz
rosc     = 5328kHz
rosc     = 5325kHz
```

Clock - RP2040

3. PLL (*Phase-Locked Loop*) - 2.18. :

- ✓ O PLL (*Phase-Locked Loop*) é um componente que permite ao RP2040 aumentar a frequência do *clock* base (XOSC) para valores maiores, como 125 MHz ou até mais.

- ✓ Ele faz isso multiplicando a frequência base de acordo com um fator de multiplicação definido pelo usuário.



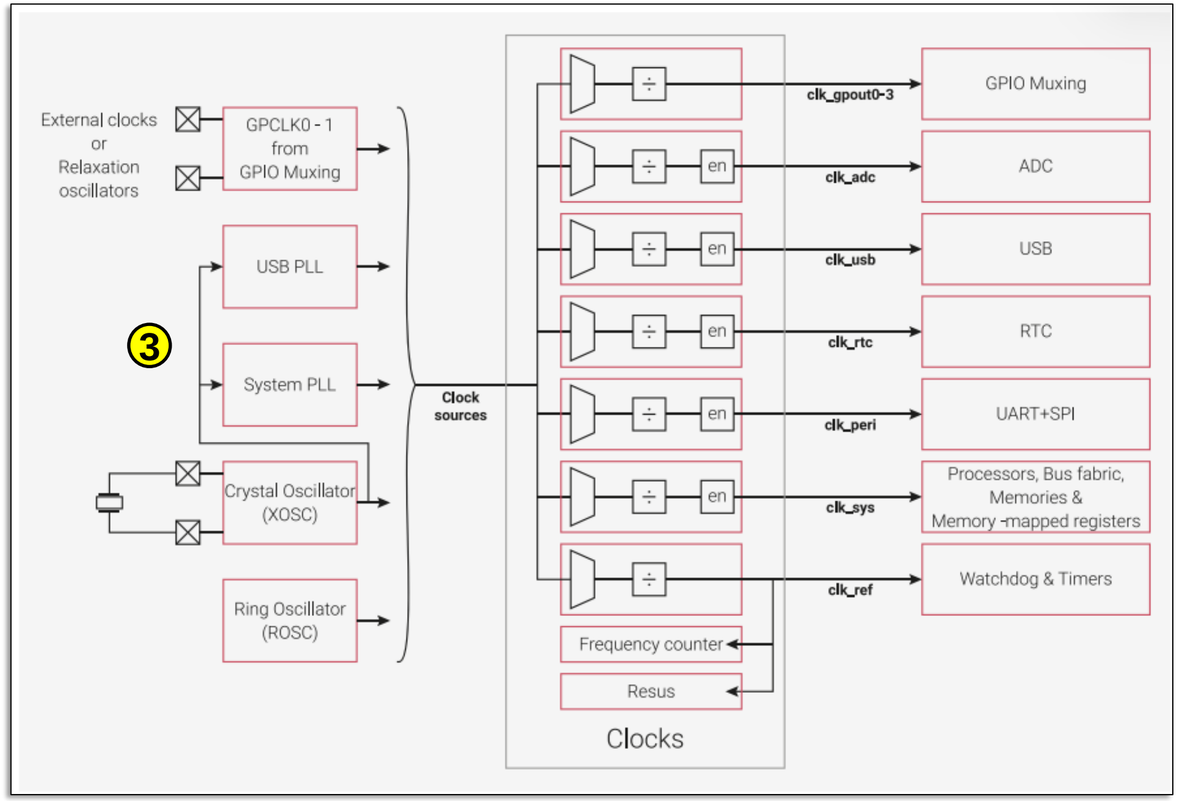
Clock - RP2040

3. PLL (*Phase-Locked Loop*) - 2.18.:

✓ Existem dois PLLs no RP2040. São eles:

1. *pll_sys* - Usado para gerar um relógio de sistema de até 133MHz.

2. *pll_usb* - Usado para gerar um relógio de referência USB de 48MHz.

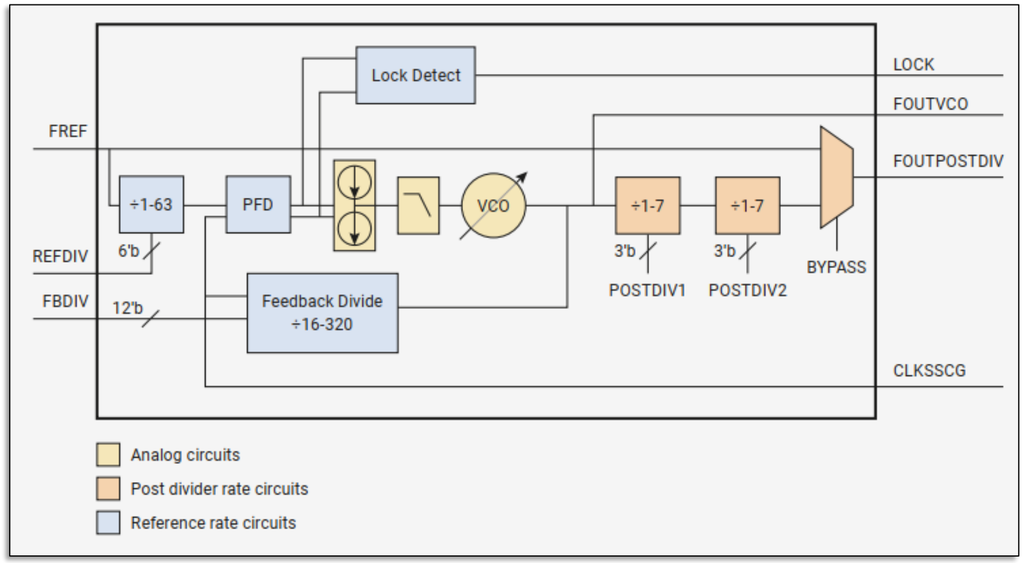


Clock - RP2040

3. PLL (*Phase-Locked Loop*) - 2.18.1:

- ✓ O clock de referência é multiplicado por um VCO (oscilador controlado por tensão) com um circuito de feedback. O VCO deve funcionar a frequências elevadas (entre 750 e 1600MHz).

- ✓ Finalmente, existem dois divisores, conhecidos como pós-divisores, que podem dividir a frequência do VCO antes de esta ser distribuída internamente.

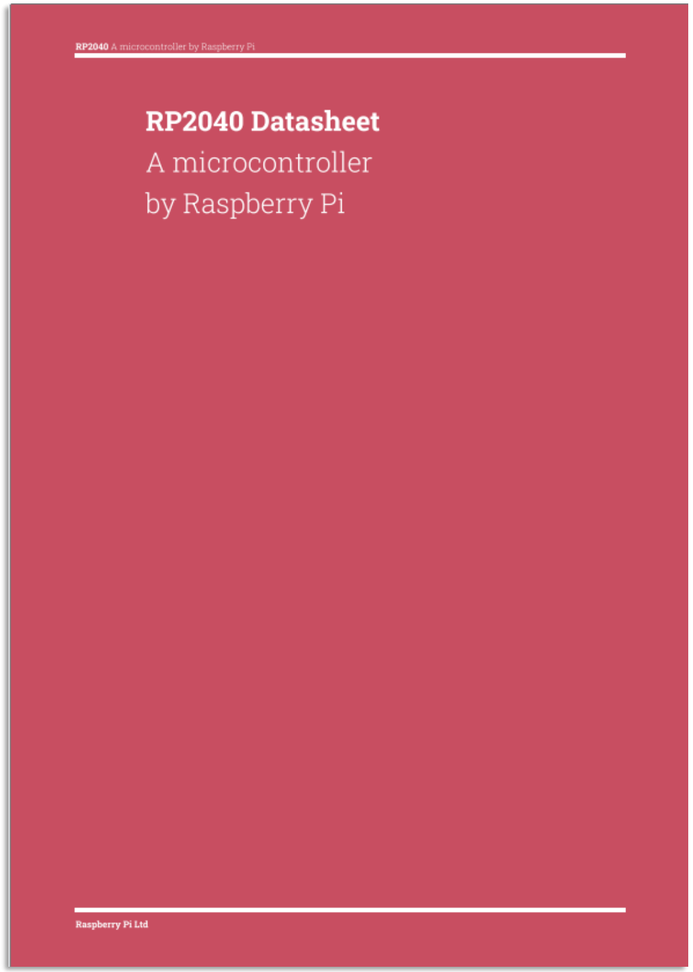


Temporizadores - RP2040

Temporizadores - RP2040

- Visão geral - 4.1.6:

✓ O periférico temporizador do RP2040 fornece uma **base de tempo global em microssegundos** para o sistema e gera interrupções com base nesta mesma base de tempo.



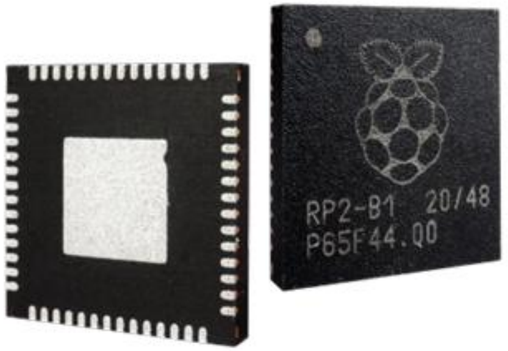
Temporizadores - RP2040

- Suporta as seguintes funcionalidades - 4.1.6:

✓ Um único contador de 64 bits, incrementado uma vez por microssegundo.

✓ **Contador (*counter*):** Possibilita a leitura de quanto tempo já se passou (64 bits -> milhares anos!).

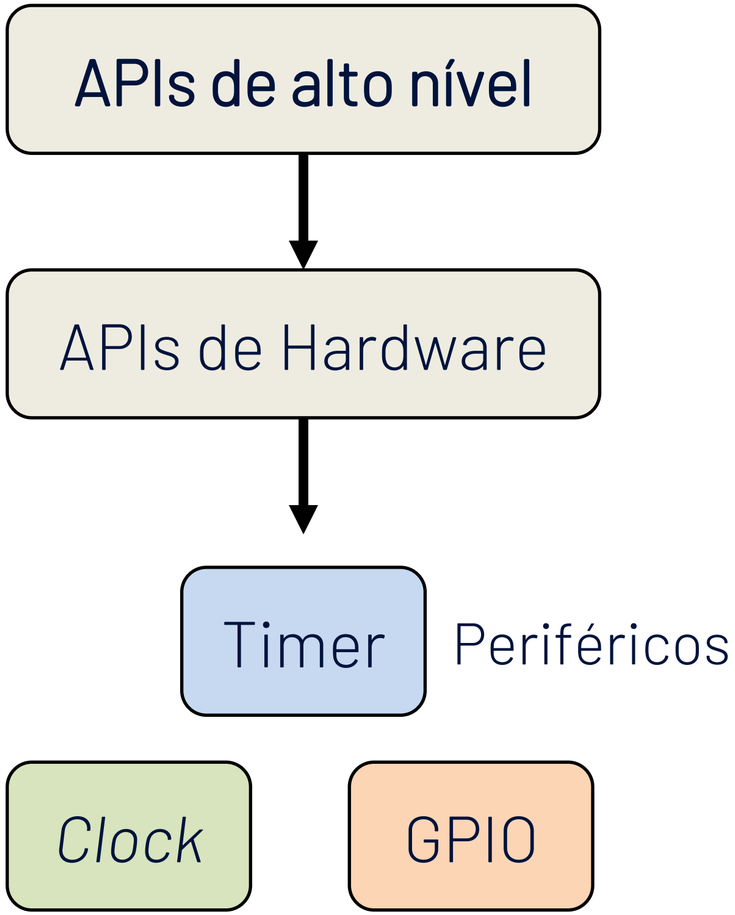
✓ **Alarme (*alarm*):** Possibilita contarmos tempo dado uma base (32bits -> tempo máximo 72 minutos)



Temporizadores - RP2040

- ✓ A base de tempo da rp2040 é de *1us* e o temporizador possui 4 alarmes e emite uma interrupção separada para cada alarme.

- ✓ Iremos acessar o timer usando a **biblioteca de alto nível** da rp2040, disponível no Kit de Desenvolvimento de Software - Pico SDK.



Temporizadores - RP2040

- Tipos de temporizadores:

✓ **Temporizador de *hardware***: Podem ser usados para gerar interrupções temporizadas ou medir o tempo com precisão.

✓ **Temporizador de *sistema (SysTick)***: É utilizado para criar intervalos regulares para execução de tarefas.

✓ **Temporizador PWM:** Permitem gerar sinais PWM com precisão.

✓ **Temporizador de contagem:** Pode ser configurado para contar eventos ou pulsos.

Temporizadores - RP2040

- Vantagens da temporização de hardware:

✓ **Menor Uso de CPU**: O temporizador está sendo gerenciado por hardware, então o loop principal não precisa fazer verificações contínuas de tempo.

✓ **Desacoplamento do Loop Principal**: A lógica de temporização está separada do loop principal. Isso significa que o loop pode ser usado para outras tarefas sem precisar lidar diretamente com o tempo.

✓ **Interrupções:** Usar temporizadores permite responder a eventos de forma mais eficiente.

Temporizadores - RP2040

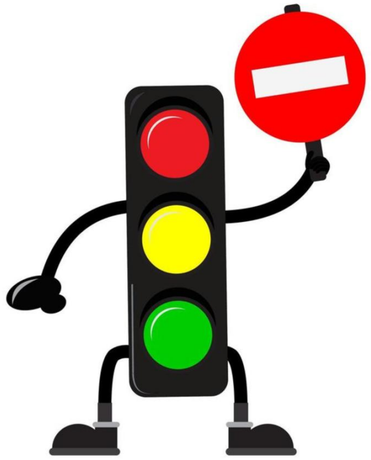
- No contexto desta aula, faremos seguintes abordagens:

✓ Ajuste da frequência de *clock* principal;

✓ Obtenção do tempo absoluto;

✓ Ajuste de Timer repetitivo;

✓ Ajuste de Timer de um disparo.

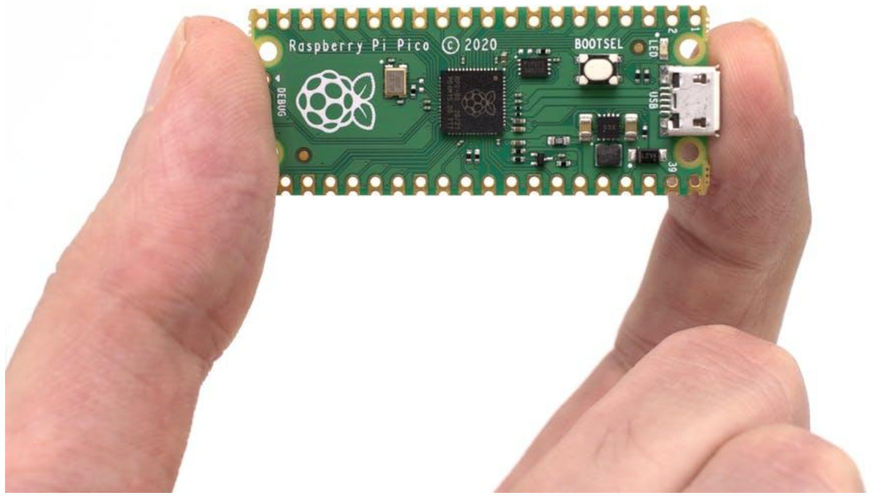


Material complementar

Exemplos de aplicação

Vamos para o microcontrolador!

Exemplos 1.1, 1.2 e 1,3



Clock do RP2040 – Exemplo 1.1

Lembrete: Organização básica do código fonte.

```
1    #include <stdio.h>
2    #include "pico/stdlib.h" //biblioteca SDK padrão do PICO - funcionalidades para programação básica.
3
4    // Definição funções, macros ou variáveis/constantes globais (1)
5
6    void main() // função principal
7    {
8         // inicializações de hardware e software (2)
9
10        while(true) // loop infinito
11        {
12             /* tarefas a serem executadas pelo microcontrolador durante o funcionamento */ (3)
13        }
14   }
```

Clock do RP2040 – Exemplo 1.1

Código fonte.

```c
#include <stdio.h> //Biblioteca padrão do C
#include "pico/stdlib.h"     // Biblioteca padrão para funções básicas de E/S.
#include "hardware/clocks.h" // Biblioteca para configuração e gerenciamento dos clocks do microcontrolador.
#include "hardware/pll.h"    // Biblioteca para controle e configuração dos Phase-Locked Loops (PLLs).

//função para obter as frequências de clock presentes no rp2040
void measure_freqs(void) {
    uint f_pll_sys = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_PLL_SYS_CLKSRC_PRIMARY);
    uint f_pll_usb = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_PLL_USB_CLKSRC_PRIMARY);
    uint f_rosc = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_ROSC_CLKSRC);
    uint f_clk_sys = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_CLK_SYS);
    uint f_clk_peri = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_CLK_PERI);
    uint f_clk_usb = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_CLK_USB);
    uint f_clk_adc = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_CLK_ADC);
    uint f_clk_rtc = frequency_count_khz(CLOCKS_FC0_SRC_VALUE_CLK_RTC);

    printf("pll_sys  = %dkHz\n", f_pll_sys);
    printf("pll_usb  = %dkHz\n", f_pll_usb);
    printf("rosc     = %dkHz\n", f_rosc);
    printf("clk_sys  = %dkHz\n", f_clk_sys);
    printf("clk_peri = %dkHz\n", f_clk_peri);
    printf("clk_usb  = %dkHz\n", f_clk_usb);
    printf("clk_adc  = %dkHz\n", f_clk_adc);
    printf("clk_rtc  = %dkHz\n", f_clk_rtc);
}

int main() //função principal
{
        // Inicializa a comunicação serial, permitindo o uso das funções padrão de E/S como printf.
        stdio_init_all();

        //coloca a frequência de clock para 100 MHz
        set_sys_clock_khz(100000, false);

        while (true)
        {
                printf("imprimindo frequências\r\n");
                measure_freqs(); //imprimir frequências
                sleep_ms(5000); //delay
        }
}
```

Porta Serial.

```
imprimindo frequências:
pll_sys  = 100000kHz
pll_usb  = 48000kHz
rosc     = 5341kHz
clk_sys  = 100000kHz
clk_peri = 48000kHz
clk_usb  = 48000kHz
clk_adc  = 48000kHz
clk_rtc  = 47kHz
```

Cod: freq_01

GitHub

Tempo absoluto – Exemplo 1.2

Código fonte.

```c
#include <stdio.h>
#include "pico/stdlib.h"

int main() {

    stdio_init_all();

    while(1){

        uint32_t start_ms = to_ms_since_boot(get_absolute_time());


        sleep_ms(1000);
        printf("tempo absoluto em ms: %d \n", start_ms);
    }
}
```

Porta Serial – valores em ms.

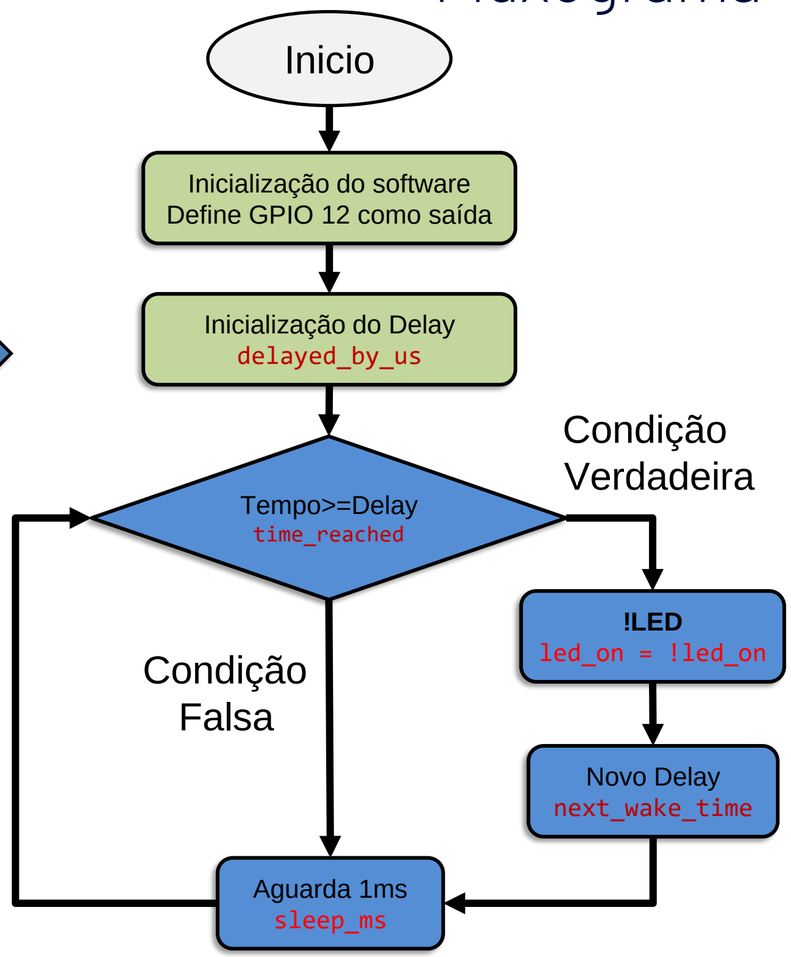
```
---- Porta serial reaberta COM21 ----
tempo absoluto em ms: 0
tempo absoluto em ms: 1001
tempo absoluto em ms: 2001
tempo absoluto em ms: 3001
tempo absoluto em ms: 4001
tempo absoluto em ms: 5001
tempo absoluto em ms: 6001
tempo absoluto em ms: 7001
tempo absoluto em ms: 8001
tempo absoluto em ms: 9001
tempo absoluto em ms: 10001
tempo absoluto em ms: 11001
tempo absoluto em ms: 12001
tempo absoluto em ms: 13001
```

Temporizador de sistema – Exemplo 1.3

Código fonte.

```c
#include <stdio.h>
#include "pico/stdlib.h" // Biblioteca padrão para funcionalidades básicas como GPIO, temporização e E/S.
#include "pico/time.h"   // Biblioteca para gerenciamento de tempo, como manipulação de temporizadores e atrasos.

#define LED_PIN_RED 12
bool led_on = false;

int main() {

    stdio_init_all(); // Inicializa a comunicação serial
    // Inicializar o pino GPIO11
    gpio_init(LED_PIN_RED);
    gpio_set_dir(LED_PIN_RED,true);
    // Define um intervalo de tempo em milissegundos
    uint32_t interval = 1000;
    // Calcula o próximo tempo absoluto em que a ação deve ocorrer.
    // get_absolute_time() retorna o tempo atual do sistema.
    // delayed_by_us() calcula um tempo futuro adicionando o intervalo em microsegundos ao tempo atual.

    absolute_time_t next_wake_time = delayed_by_us(get_absolute_time(), interval * 1000);

    // Loop infinito que mantém o programa em execução contínua.
    while (true) {
        // Verifica se o tempo atual atingiu ou ultrapassou o tempo definido em next_wake_time.
        if (time_reached(next_wake_time)) {
            // Imprime uma mensagem na saída serial indicando que 1 segundo se passou.
            printf("1 segundo passou\n");
            //Liga ou desliga o led.
            led_on = !led_on;
            gpio_put(LED_PIN_RED,led_on);
            // Calcula o próximo tempo de despertar, adicionando o intervalo de 1 segundo ao tempo atual.
            // Isso garante que a próxima execução aconteça exatamente 1 segundo após a última.
            next_wake_time = delayed_by_us(next_wake_time, interval * 1000);
        }
        // Introduz uma pequena pausa de 1 milissegundo para reduzir o uso da CPU.
        // Isso evita que o loop rode desnecessariamente rápido, economizando recursos.
        sleep_ms(1);
    }
}
```

Fluxograma



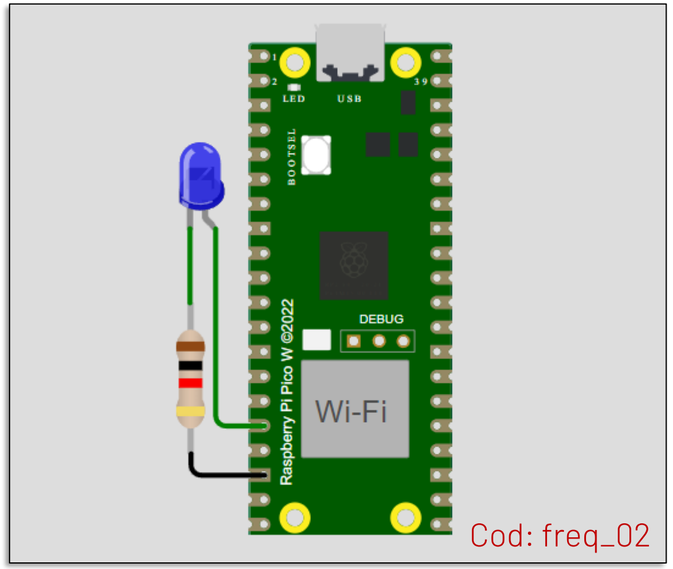
Objetivo: rotina para piscar o LED (GPIO 12).

Temporizador de sistema – Exemplo 1.3

Código fonte.

```c
#include <stdio.h>
#include "pico/stdlib.h" // Biblioteca padrão para funcionalidades básicas como GPIO, temporização e E/S.
#include "pico/time.h"   // Biblioteca para gerenciamento de tempo, como manipulação de temporizadores e atrasos.

#define LED_PIN_RED 12
bool led_on = false;

int main() {

    stdio_init_all(); // Inicializa a comunicação serial
    // Inicializar o pino GPIO11
    gpio_init(LED_PIN_RED);
    gpio_set_dir(LED_PIN_RED,true);
    // Define um intervalo de tempo em milissegundos
    uint32_t interval = 1000;
    // Calcula o próximo tempo absoluto em que a ação deve ocorrer.
    // get_absolute_time() retorna o tempo atual do sistema.
    // delayed_by_us() calcula um tempo futuro adicionando o intervalo em microsegundos ao tempo atual.

    absolute_time_t next_wake_time = delayed_by_us(get_absolute_time(), interval * 1000);

    // Loop infinito que mantém o programa em execução contínua.
    while (true) {
        // Verifica se o tempo atual atingiu ou ultrapassou o tempo definido em next_wake_time.
        if (time_reached(next_wake_time)) {
            // Imprime uma mensagem na saída serial indicando que 1 segundo se passou.
            printf("1 segundo passou\n");
            //Liga ou desliga o led.
            led_on = !led_on;
            gpio_put(LED_PIN_RED,led_on);
            // Calcula o próximo tempo de despertar, adicionando o intervalo de 1 segundo ao tempo atual.
            // Isso garante que a próxima execução aconteça exatamente 1 segundo após a última.
            next_wake_time = delayed_by_us(next_wake_time, interval * 1000);
        }
        // Introduz uma pequena pausa de 1 milissegundo para reduzir o uso da CPU.
        // Isso evita que o loop rode desnecessariamente rápido, economizando recursos.
        sleep_ms(1);
    }
}
```

Simulação Wokwi.

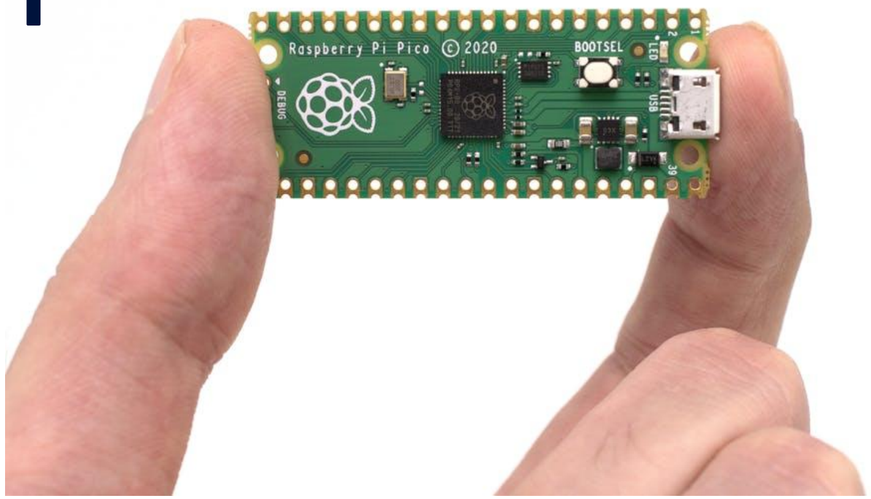


GitHub

Objetivo: rotina para piscar o LED (GPIO 12).

Vamos para o microcontrolador!

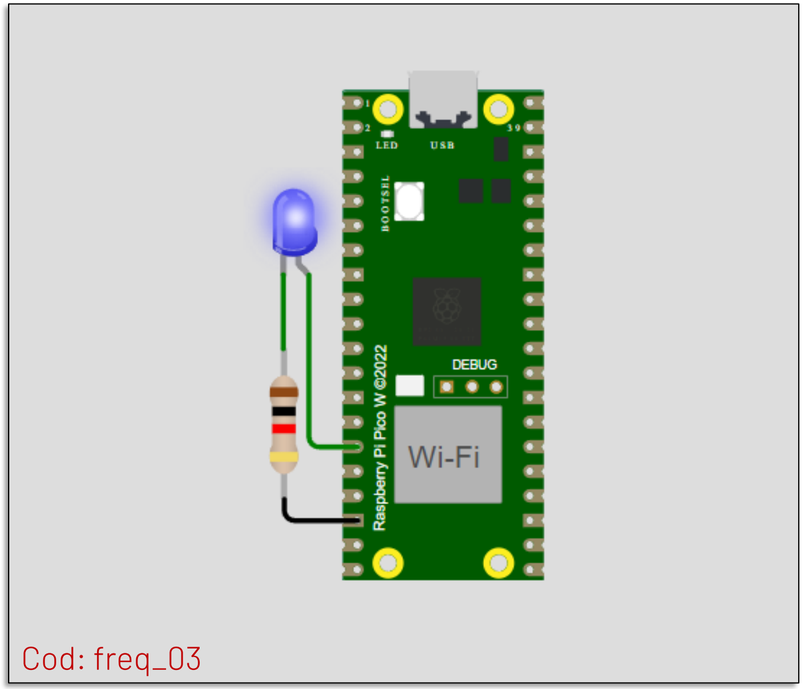
Exemplos 2.1 e 2.2
Atividade 1

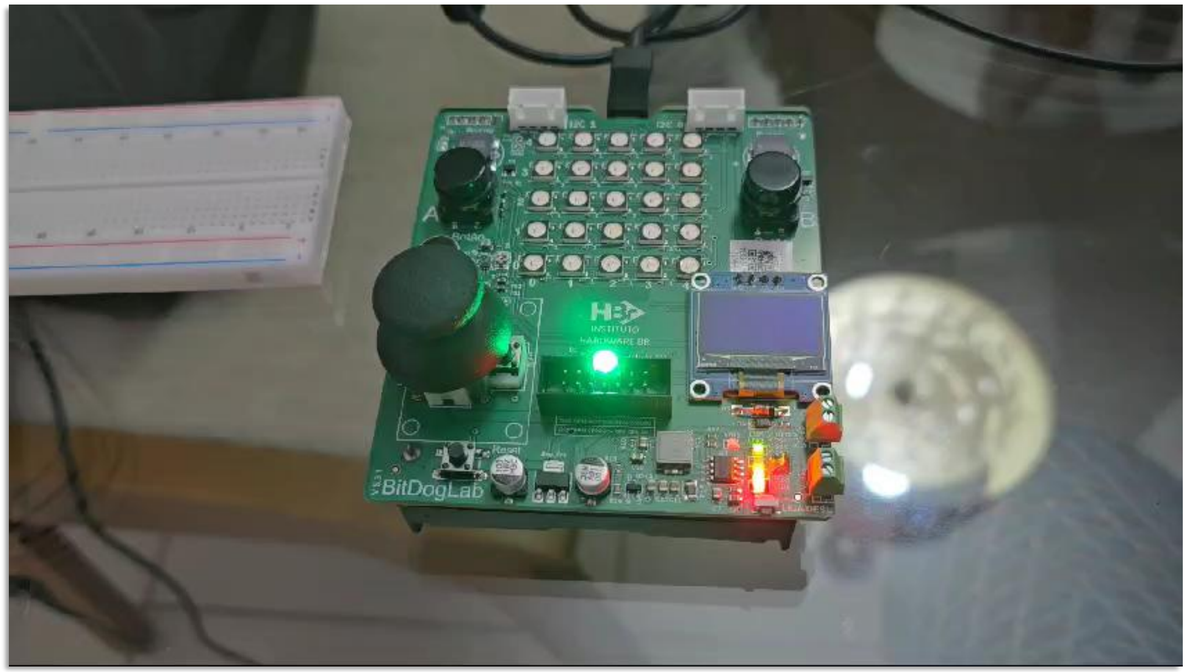


Temporização periódica – Exemplo 2.1

Objetivo: Através da temporização de *Hardware*, criar código para piscar um LED.

Simulação Wokwi





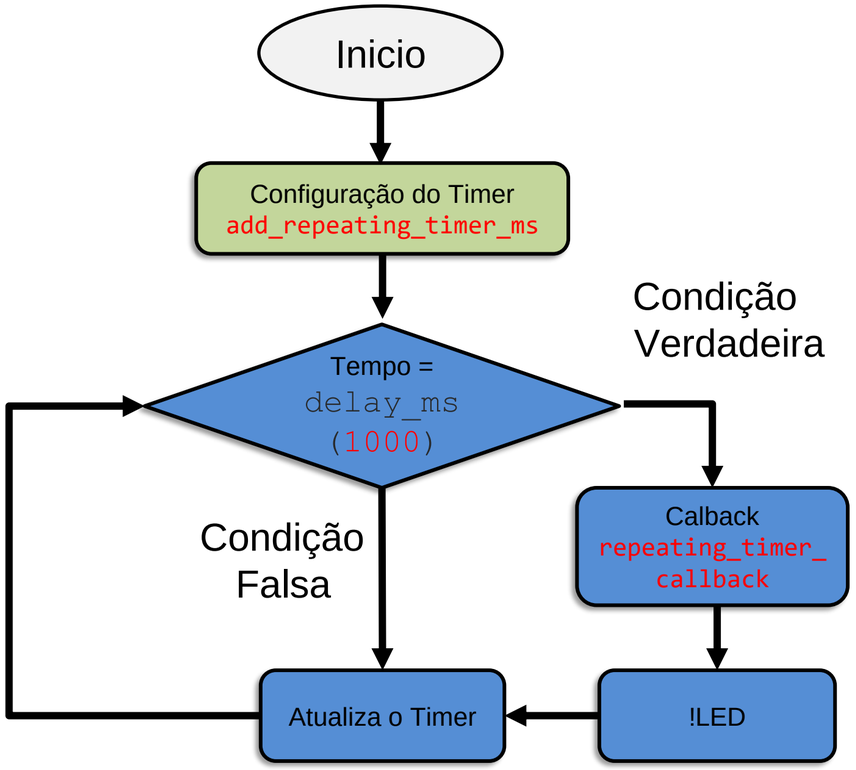
Experimento: BitDogLab – GPIO 11.

Exemplo 2.1

Código fonte.

```c
#include <stdio.h>
#include "pico/stdlib.h"   // Inclui a biblioteca padrão para funcionalidades básicas como GPIO.
#include "hardware/timer.h" // Inclui a biblioteca para gerenciamento de temporizadores de hardware.

#define LED_PIN_RED 11
bool led_on = false;

// Função de callback que será chamada repetidamente pelo temporizador
// O tipo bool indica que a função deve retornar verdadeiro ou falso para continuar ou parar o temporizador.
bool repeating_timer_callback(struct repeating_timer *t) {
    // Imprime uma mensagem na saída serial indicando que 1 segundo se passou.
    printf("1 segundo passou\n");
    //Liga ou desliga o led.
    led_on = !led_on;
    gpio_put(LED_PIN_RED,led_on);
    // Retorna true para manter o temporizador repetindo. Se retornar false, o temporizador para.
    return true;
}

int main() {
    // Inicializa a comunicação serial, permitindo o uso de funções como printf.
    stdio_init_all();

    // Inicializar o pino GPIO11
    gpio_init(LED_PIN_RED);
    gpio_set_dir(LED_PIN_RED,true);

    // Declaração de uma estrutura de temporizador de repetição.
    // Esta estrutura armazenará informações sobre o temporizador configurado.
    struct repeating_timer timer;

    // Configura o temporizador para chamar a função de callback a cada 1 segundo.
    add_repeating_timer_ms(1000, repeating_timer_callback, NULL, &timer);

    // Loop infinito que mantém o programa em execução.
    while (true) {
        // Pausa de 10 segundos para reduzir o uso da CPU.
        sleep_ms(10000);
        printf("rotina de repetição\n");
    }
    return 0;
}
```

Fluxograma do timer

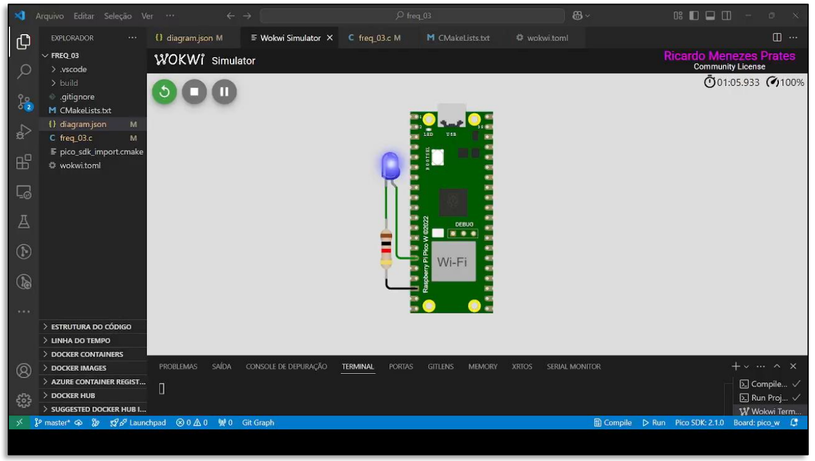


Exemplo 2.1

Código fonte.

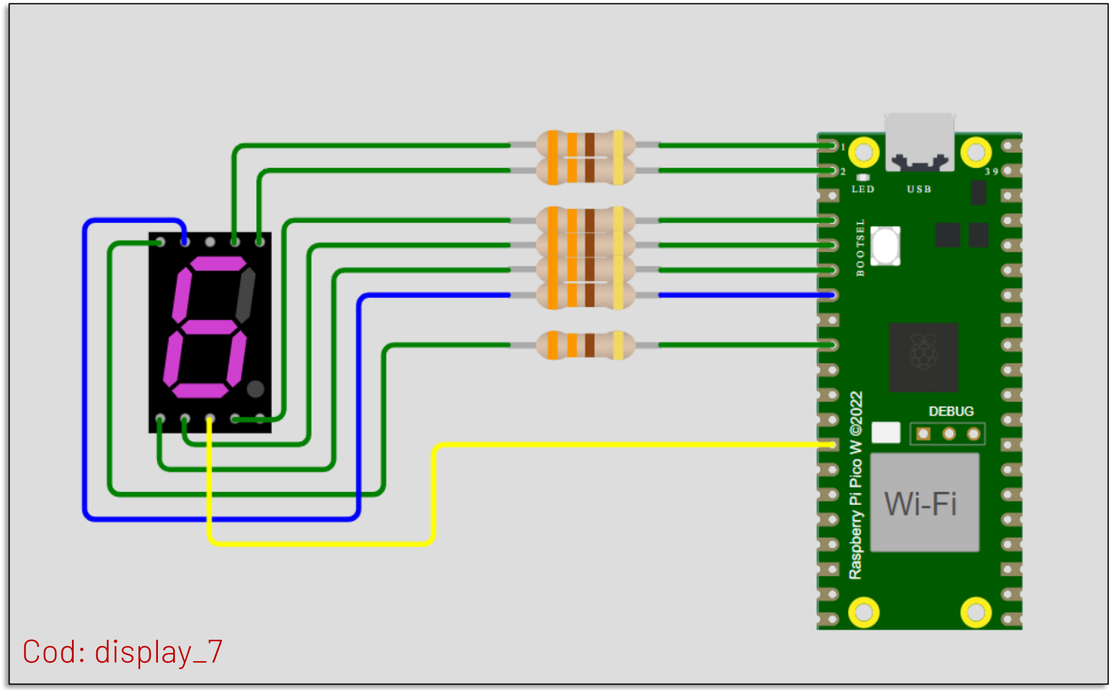
```c
#include <stdio.h>
#include "pico/stdlib.h"   // Inclui a biblioteca padrão para funcionalidades básicas como GPIO.
#include "hardware/timer.h" // Inclui a biblioteca para gerenciamento de temporizadores de hardware.

#define LED_PIN_RED 11
bool led_on = false;

// Função de callback que será chamada repetidamente pelo temporizador
// O tipo bool indica que a função deve retornar verdadeiro ou falso para continuar ou parar o temporizador.
bool repeating_timer_callback(struct repeating_timer *t) {
    // Imprime uma mensagem na saída serial indicando que 1 segundo se passou.
    printf("1 segundo passou\n");
    //Liga ou desliga o led.
    led_on = !led_on;
    gpio_put(LED_PIN_RED,led_on);
    // Retorna true para manter o temporizador repetindo. Se retornar false, o temporizador para.
    return true;
}

int main() {
    // Inicializa a comunicação serial, permitindo o uso de funções como printf.
    stdio_init_all();

    // Inicializar o pino GPIO11
    gpio_init(LED_PIN_RED);
    gpio_set_dir(LED_PIN_RED,true);

    // Declaração de uma estrutura de temporizador de repetição.
    // Esta estrutura armazenará informações sobre o temporizador configurado.
    struct repeating_timer timer;

    // Configura o temporizador para chamar a função de callback a cada 1 segundo.
    add_repeating_timer_ms(1000, repeating_timer_callback, NULL, &timer);

    // Loop infinito que mantém o programa em execução.
    while (true) {
        // Pausa de 10 segundos para reduzir o uso da CPU.
        sleep_ms(10000);
        printf("rotina de repetição\n");
    }
    return 0;
}
```

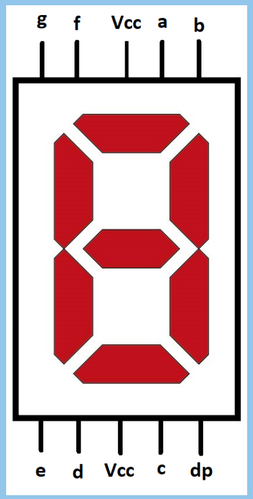
Simulação:



Temporização periódica – Exemplo 2.2

Objetivo: Criar um código para contagem de um display de 7 segmentos.

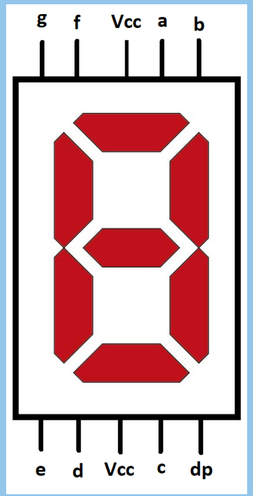


	Componentes	GPIO
1	Display 7 	a – GPIO 0 b – GPIO 1 c – GPIO 2 d – GPIO 3 e – GPIO 4 f – GPIO 5 g – GPIO 6 Com – GND4

Temporização periódica – Exemplo 2.2

Componente: Display de 7 segmentos.

Entrada (BCD/Binário)				Saídas 7 segmentos							Display
D	C	B	A	*a*	*b*	*c*	*d*	*e*	*f*	*g*	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	0	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	0	0	1	1	9

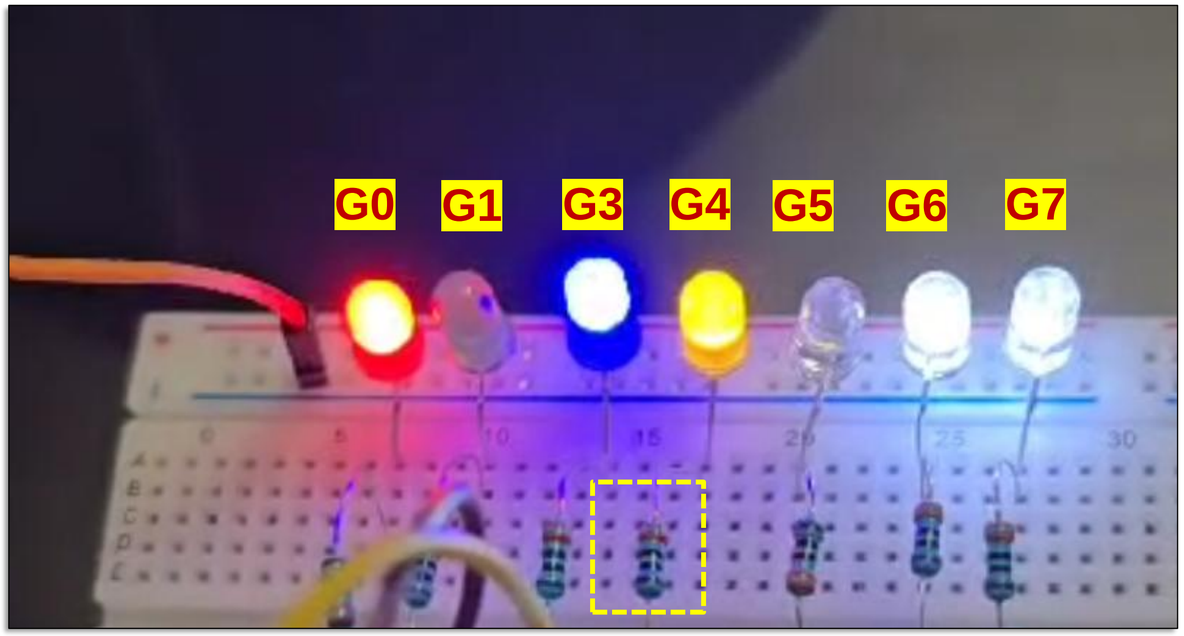
	Componentes	GPIO
1	Display 7 	a – GPIO 0 b – GPIO 1 c – GPIO 2 d – GPIO 3 e – GPIO 4 f – GPIO 5 g – GPIO 6 Com – GND4

Temporização periódica – Exemplo 2.2

Componente: Display de 7 segmentos.

Entrada (BCD/Binário)				Saídas 7 segmentos							Display
D	C	B	A	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	0	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	0	0	1	1	9

Valor 5 (0101).



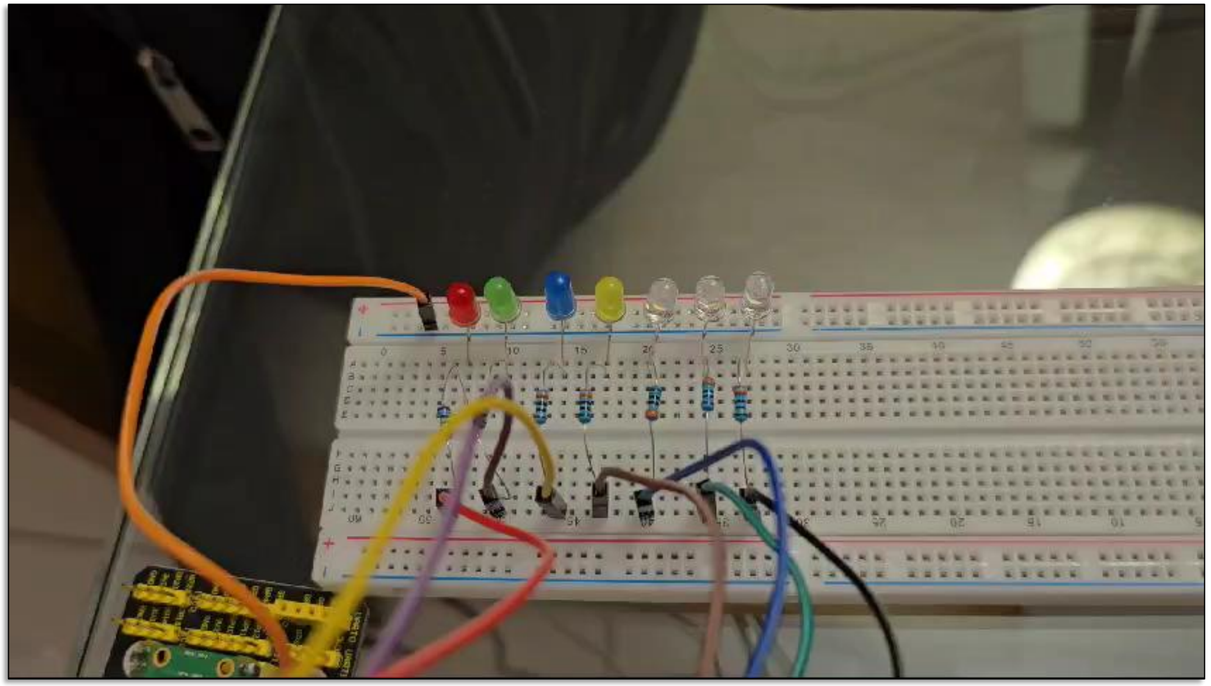
Resistor de 330 Ω

Temporização periódica – Exemplo 2.1

Componente: Display de 7 segmentos.

Entrada (BCD/Binário)				Saídas 7 segmentos							Display
D	C	B	A	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	0	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	0	0	1	1	9

Experimento com o Raspberry Pi Pico W

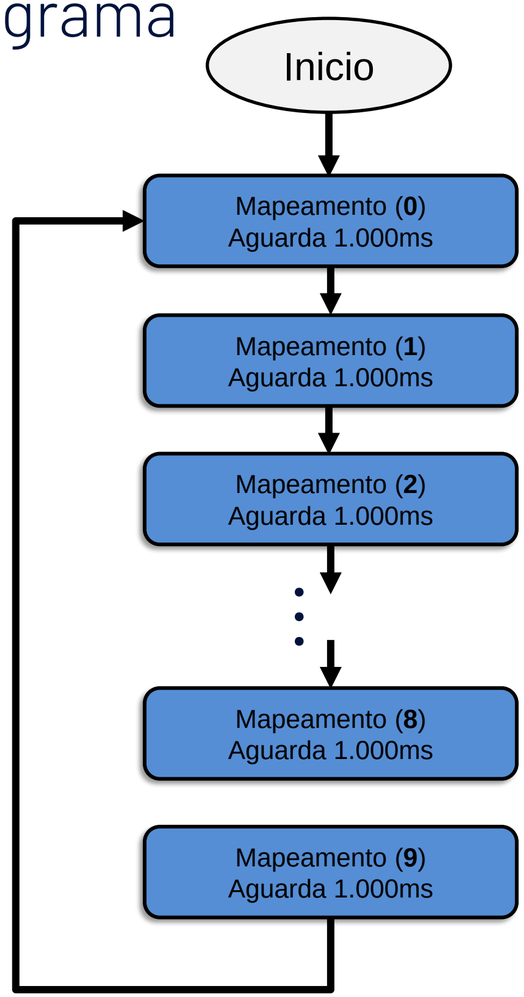


Temporização periódica – Exemplo 2.2

Trecho do código.

```c
#include <stdio.h>
#include "pico/stdlib.h"
  #include "hardware/timer.h" // Inclui a biblioteca para gerenciamento de temporizadores de hardware.

// Definição dos pinos dos segmentos do display
const uint8_t segment_pins[] = {0, 1, 2, 3, 4, 5, 6};

// Mapeamento dos dígitos para os segmentos do display
const uint8_t digits[10][7] = {
    {1, 1, 1, 1, 1, 1, 0}, // 0
    {0, 1, 1, 0, 0, 0, 0}, // 1
    {1, 1, 0, 1, 1, 0, 1}, // 2
    {1, 1, 1, 1, 0, 0, 1}, // 3
    {0, 1, 1, 0, 0, 1, 1}, // 4
    {1, 0, 1, 1, 0, 1, 1}, // 5
    {1, 0, 1, 1, 1, 1, 1}, // 6
    {1, 1, 1, 0, 0, 0, 0}, // 7
    {1, 1, 1, 1, 1, 1, 1}, // 8
    {1, 1, 1, 0, 0, 1, 1}  // 9
};

// Continua...
```

Fluxograma

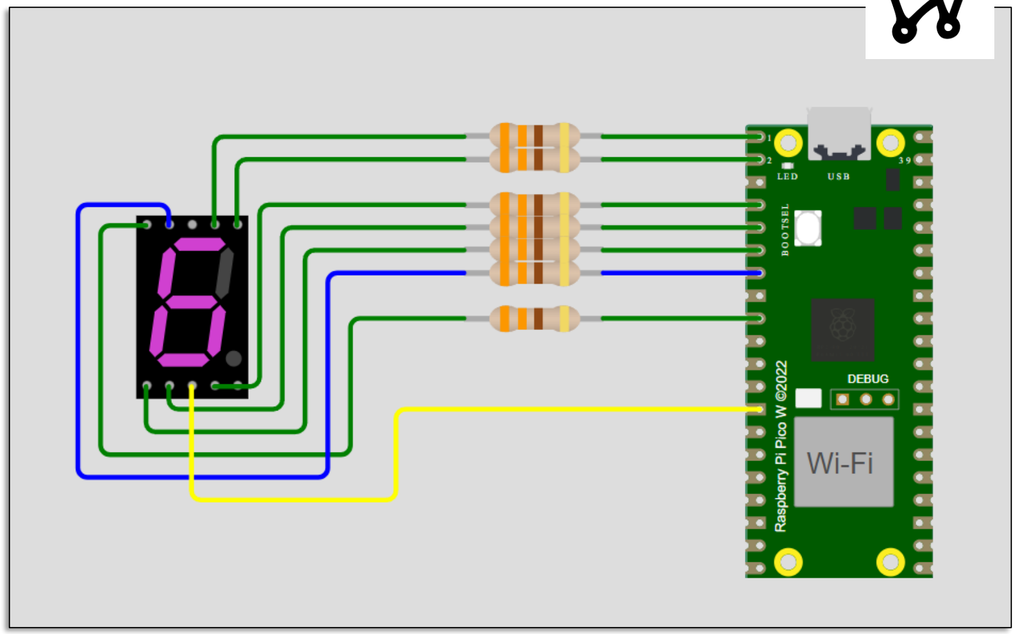


Temporização periódica – Exemplo 2.2

Trecho do código.

```c
#include <stdio.h>
#include "pico/stdlib.h"
  #include "hardware/timer.h" // Inclui a biblioteca para gerenciamento de temporizadores de hardware.

// Definição dos pinos dos segmentos do display
const uint8_t segment_pins[] = {0, 1, 2, 3, 4, 5, 6};

// Mapeamento dos dígitos para os segmentos do display
const uint8_t digits[10][7] = {
    {1, 1, 1, 1, 1, 1, 0}, // 0
    {0, 1, 1, 0, 0, 0, 0}, // 1
    {1, 1, 0, 1, 1, 0, 1}, // 2
    {1, 1, 1, 1, 0, 0, 1}, // 3
    {0, 1, 1, 0, 0, 1, 1}, // 4
    {1, 0, 1, 1, 0, 1, 1}, // 5
    {1, 0, 1, 1, 1, 1, 1}, // 6
    {1, 1, 1, 0, 0, 0, 0}, // 7
    {1, 1, 1, 1, 1, 1, 1}, // 8
    {1, 1, 1, 0, 0, 1, 1}  // 9
};

// Continua...
```

Simulação Wokwi:

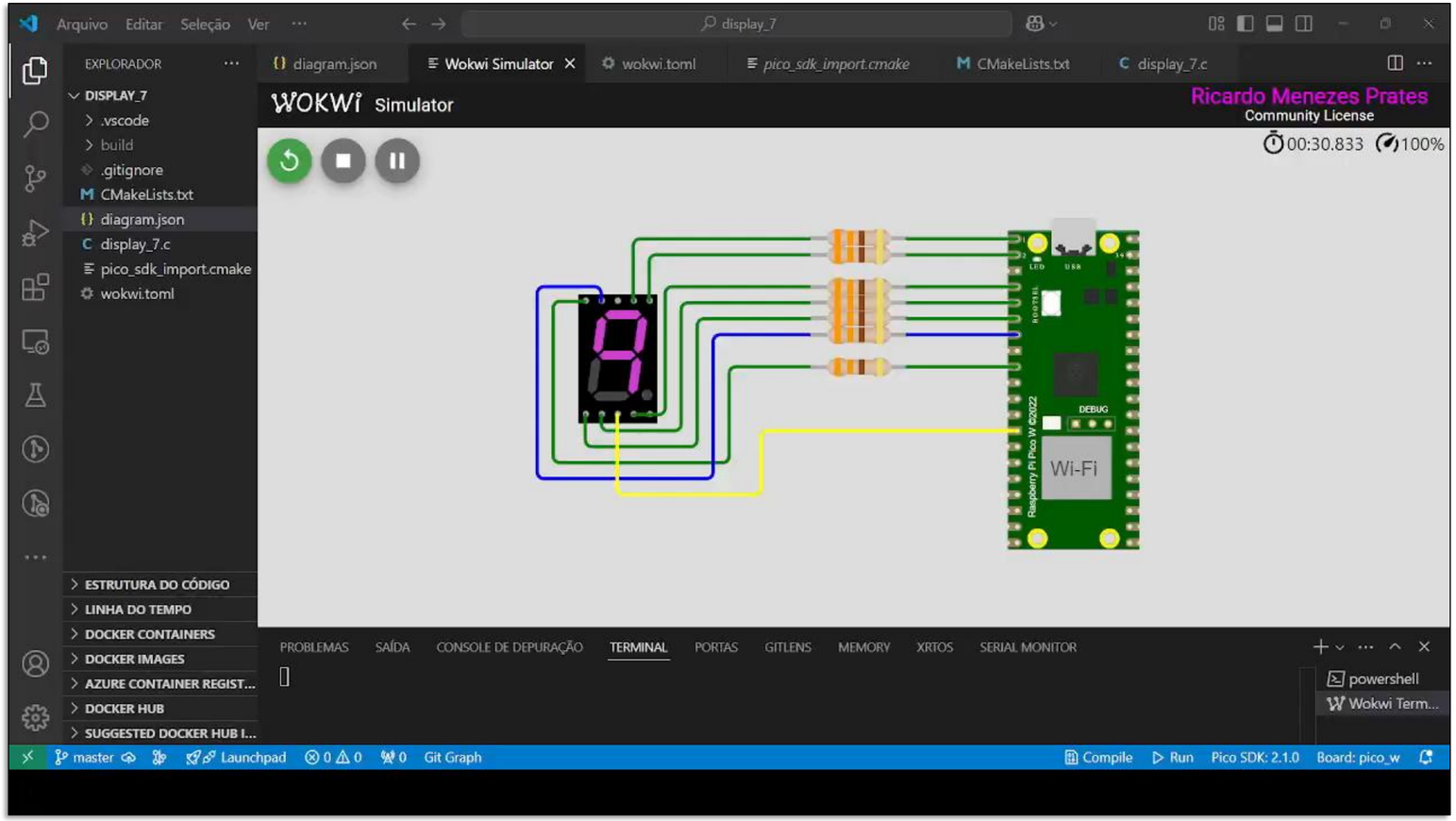


Vamos para o código!

GitHub

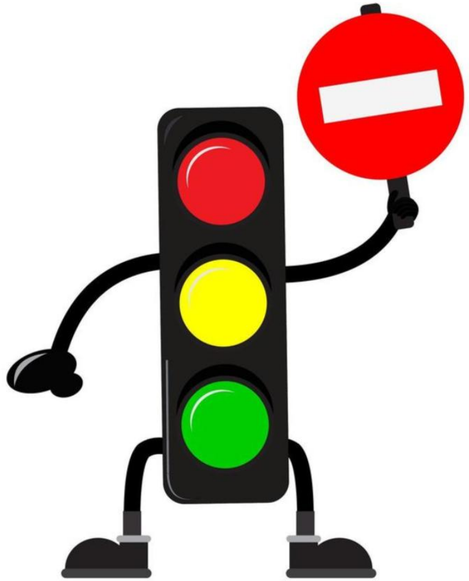
Temporização periódica – Exemplo 2

Simulação Wokwi:

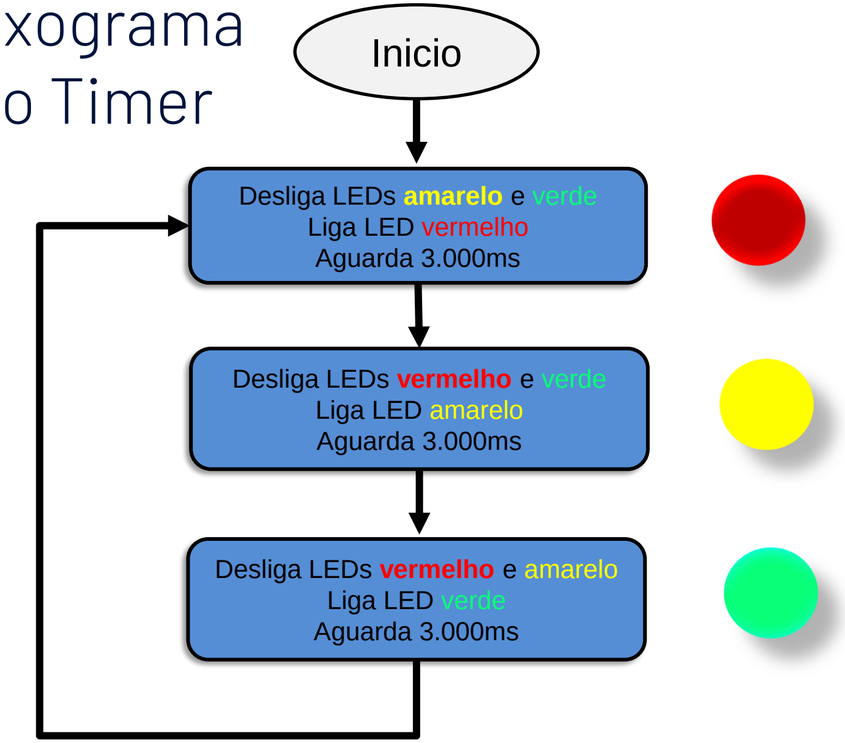


Prática com Temporizador – Atividade 1

Objetivo: Com o emprego da função "add_repeating_timer_ms()", projete um semáforo com temporização de 3 segundos para cada alteração de sinal.

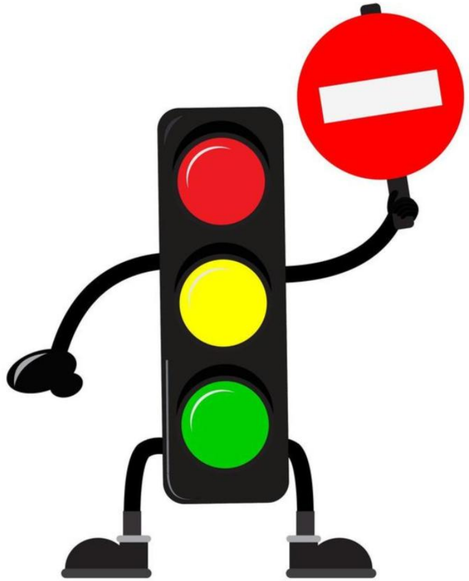


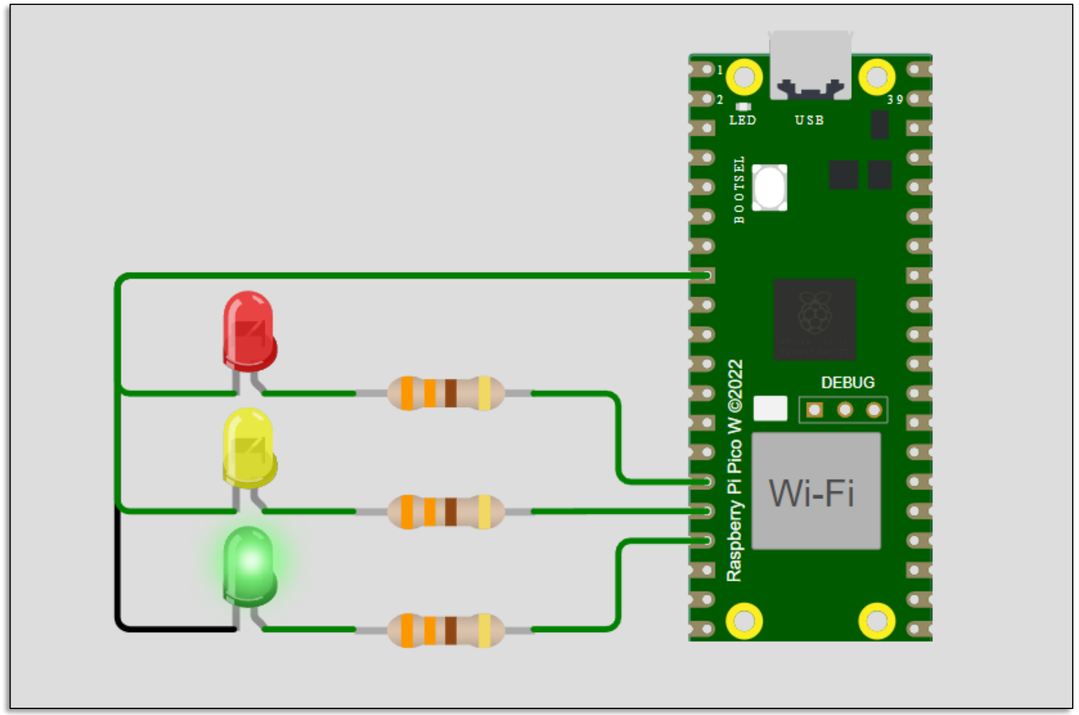
Fluxograma do Timer



Prática com Temporizador – Atividade 1

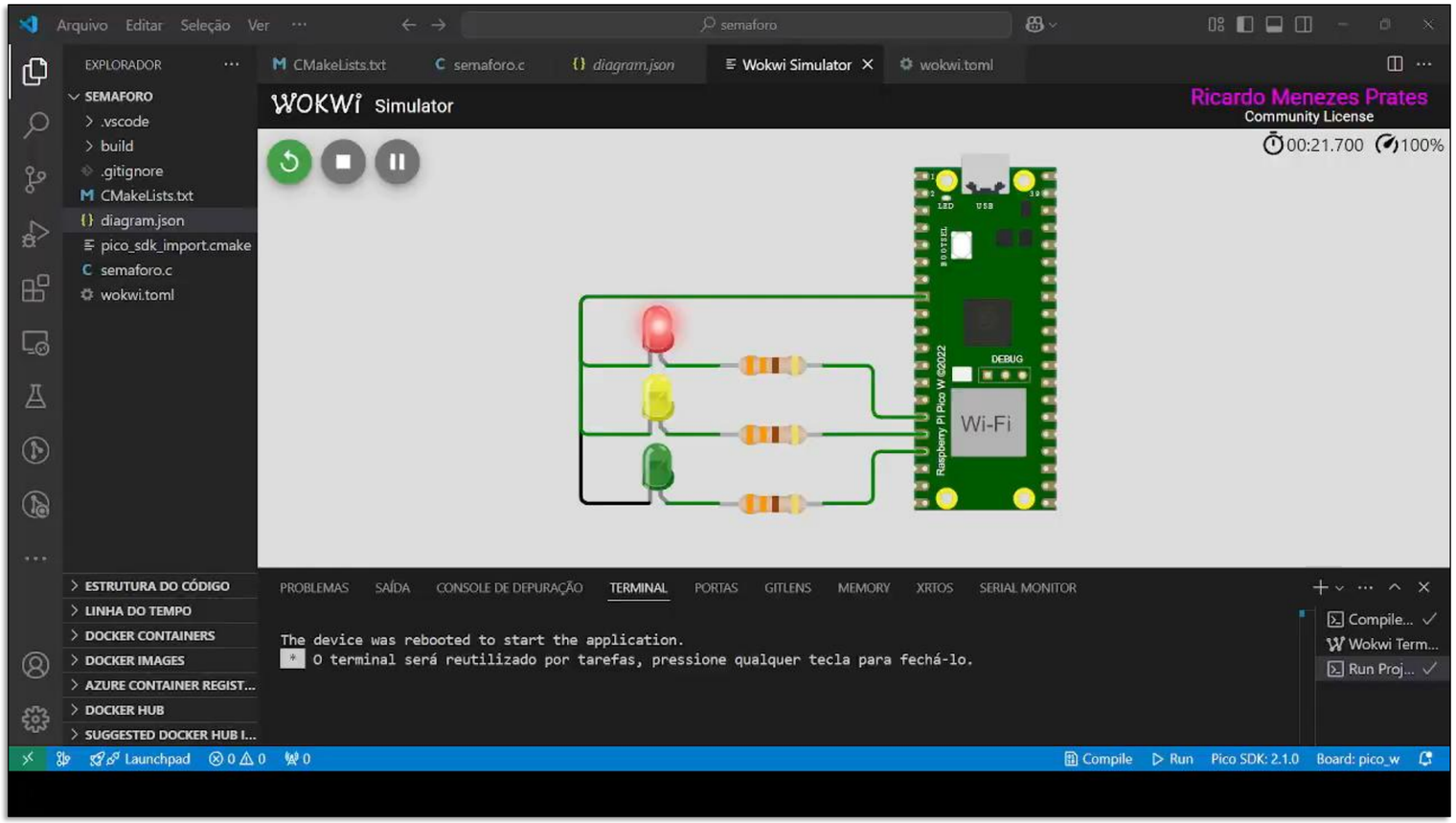
Objetivo: Com o emprego da função “add_repeating_timer_ms()”, projete um semáforo com temporização de 3 segundos para cada alteração de sinal.





Prática com Temporizador – Atividade 1

Exemplo: Veja o documento da atividade:

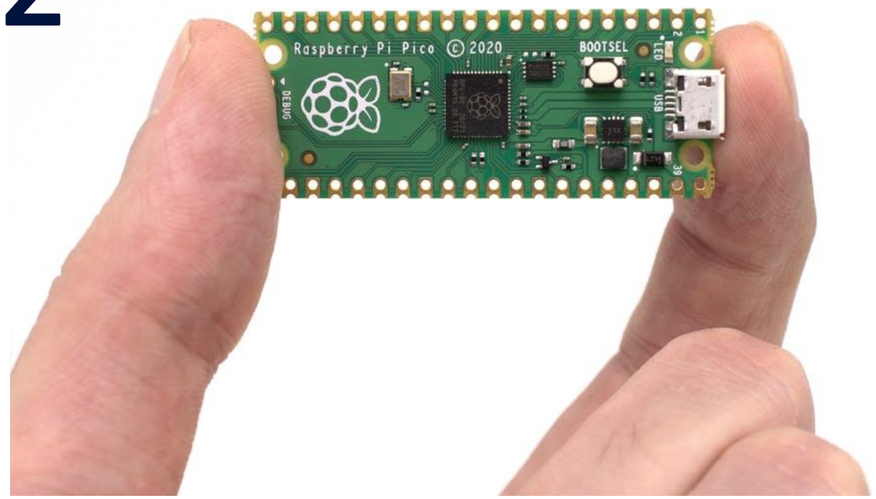


Comportamento desejado da Porta Serial.

```
Altera o sinal
Passou 1 segundo
Passou 1 segundo
Passou 1 segundo
Altera o sinal
```

Vamos para o microcontrolador!

Exemplos 3.1 e 3.2
Atividade 2



Temporização *One Shot* – Exemplo 3.1

Código fonte.

```c
#include <stdio.h>
#include "pico/stdlib.h"

// Função de callback que será chamada pelo alarme
int64_t alarm_callback(alarm_id_t id, void *user_data) {
    uint32_t start_ms = to_ms_since_boot(get_absolute_time());
    printf("tempo absoluto do alarm: %d \n", start_ms);
    // Pode retornar um valor em microssegundos para retornar no futuro.
    return 0;
}

int main() {

    stdio_init_all();

    // Chama alarm_callback com temporização de 5000 ms
    alarm_id_t alarm = add_alarm_in_ms(5000, alarm_callback, NULL, false);

    if (!alarm) {
        printf("Failed to add timer\n");
    }

    while(1){
        sleep_ms(1000);
        uint32_t start_ms = to_ms_since_boot(get_absolute_time());
        printf("tempo absoluto - rotina While: %d \n", start_ms);
    }
}
```

Porta Serial – valores em ms.

```
---- Porta serial reaberta COM21 ----
tempo absoluto - rotina While 1000
tempo absoluto - rotina While 2001
tempo absoluto - rotina While 3001
tempo absoluto - rotina While 4001
tempo absoluto do alarm: 5000
tempo absoluto - rotina While 5001
tempo absoluto - rotina While 6001
tempo absoluto - rotina While 7001
tempo absoluto - rotina While 8001
tempo absoluto - rotina While 9001
tempo absoluto - rotina While 10001
tempo absoluto - rotina While 11001
tempo absoluto - rotina While 12001
```

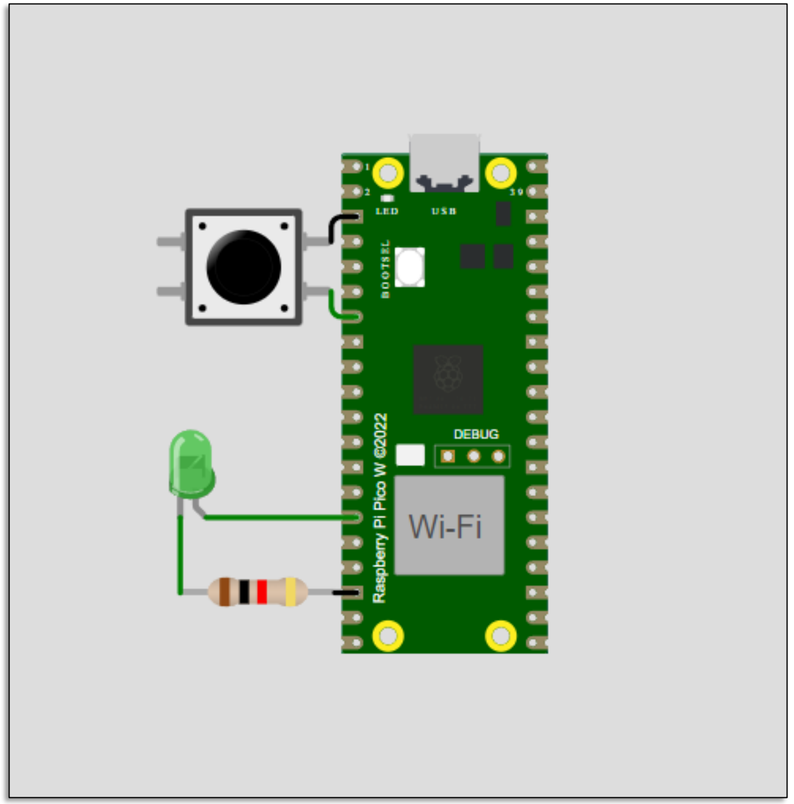
Cod: alarm_clock

GitHub

Temporização *One Shot* – Exemplo 3.2

Trecho do código fonte.

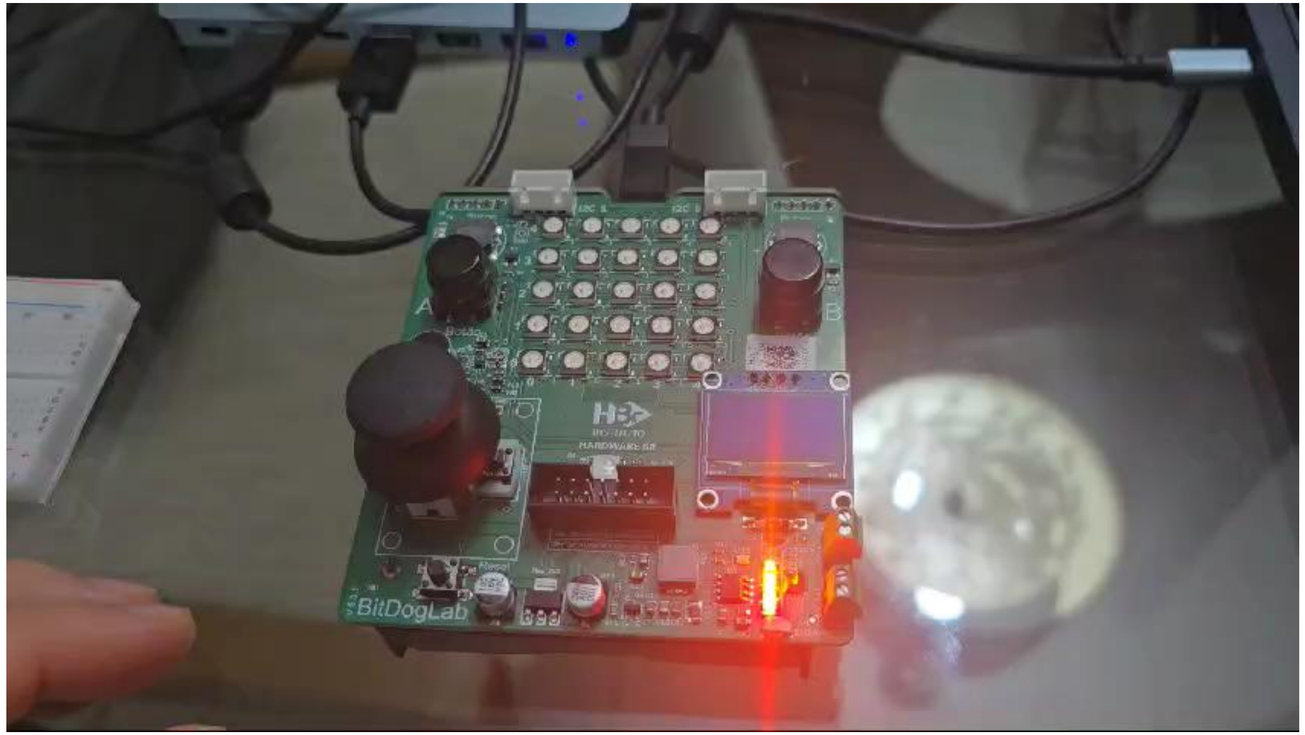
```c
#include <stdio.h> // Biblioteca padrão do C.
#include "pico/stdlib.h" // Biblioteca padrão do Raspberry Pi Pico para controle de GPIO.
#include "pico/time.h"   // Biblioteca para gerenciamento de temporizadores e alarmes.

const uint LED_PIN = 11;    // Define o pino GPIO 11 para controlar o LED.
const uint BUTTON_PIN = 5;  // Define o pino GPIO 5 para ler o estado do botão.

bool led_on = false;        // Variável global para armazenar o estado do LED (não utilizada neste código).
bool led_active = false;    // Indica se o LED está atualmente aceso (para evitar múltiplas ativações).

// Função de callback para desligar o LED após o tempo programado.
int64_t turn_off_callback(alarm_id_t id, void *user_data) {

    // Desliga o LED configurando o pino LED_PIN para nível baixo.
    gpio_put(LED_PIN, false);

    // Atualiza o estado de 'led_active' para falso, indicando que o LED está desligado.
    led_active = false;

    // Retorna 0 para indicar que o alarme não deve se repetir.
    return 0;
}
// Continua...
```



Cod: botão_timer

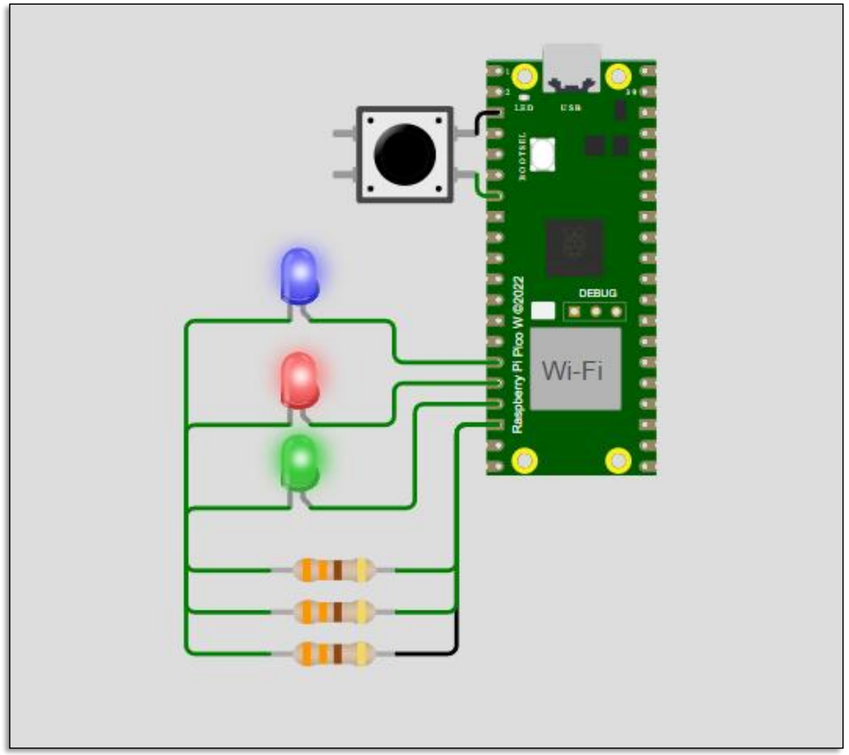
Vamos para o código!

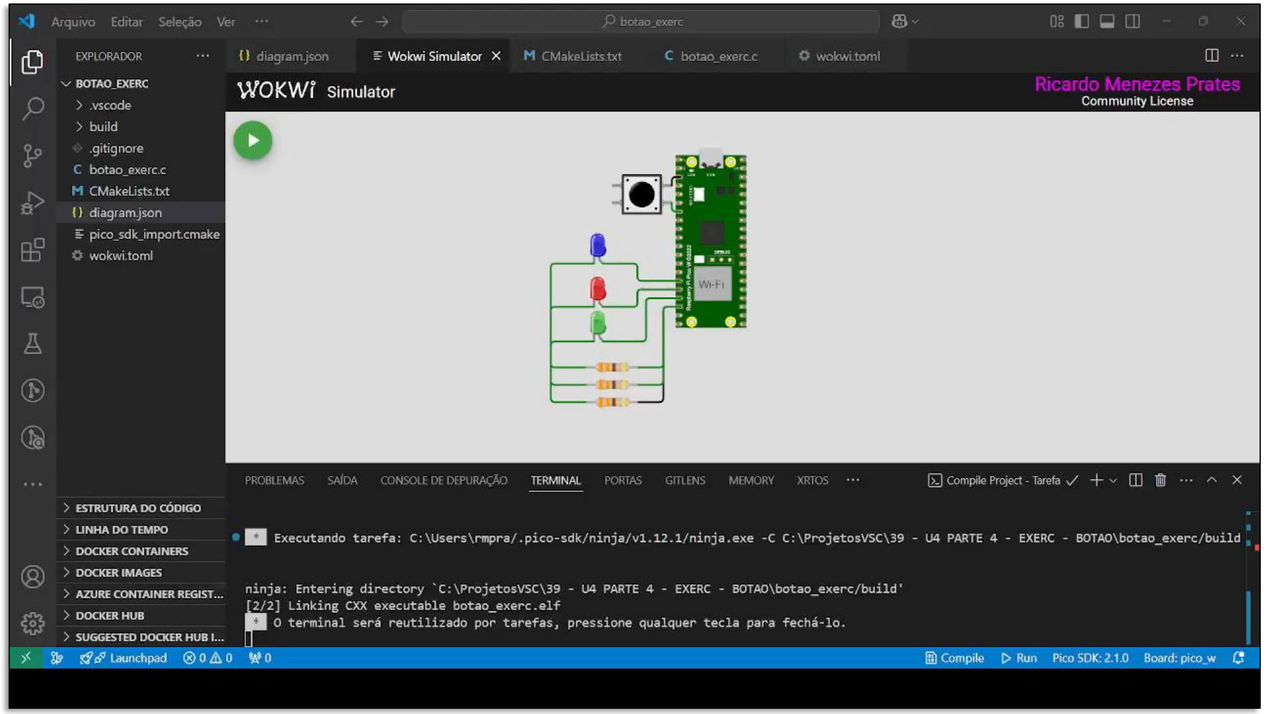
Temporização *One Shot* – Exemplo 3.2



Temporização *One Shot* – Atividade 2

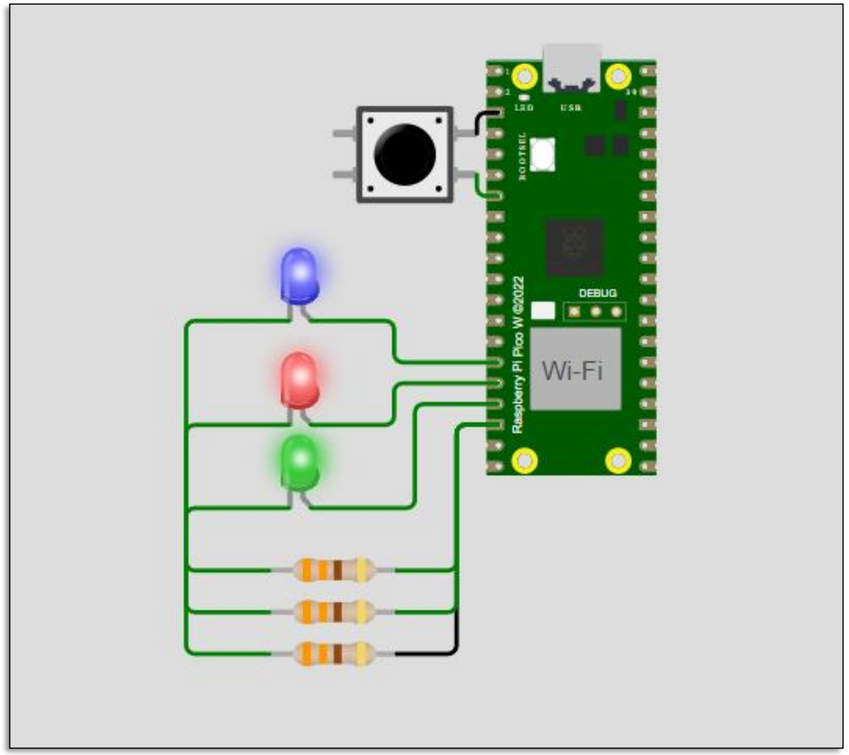
Exemplo: Veja o documento da atividade:





Temporização *One Shot* – Atividade 2

Exemplo: Veja o documento da atividade:



Conclusões

Conclusões

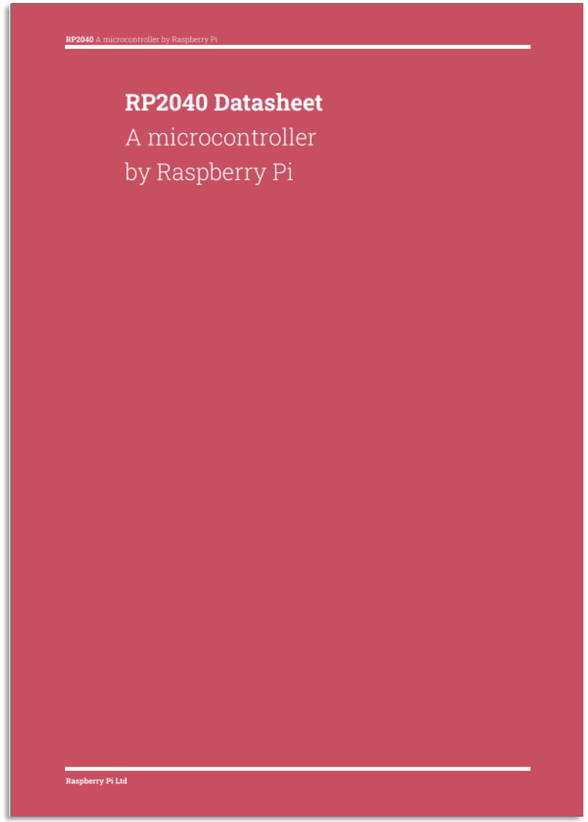
- O microcontrolador RP2040 conta com uma arquitetura de *clocks* **bastante flexível e complexa** – contando com osciladores internos e externos.

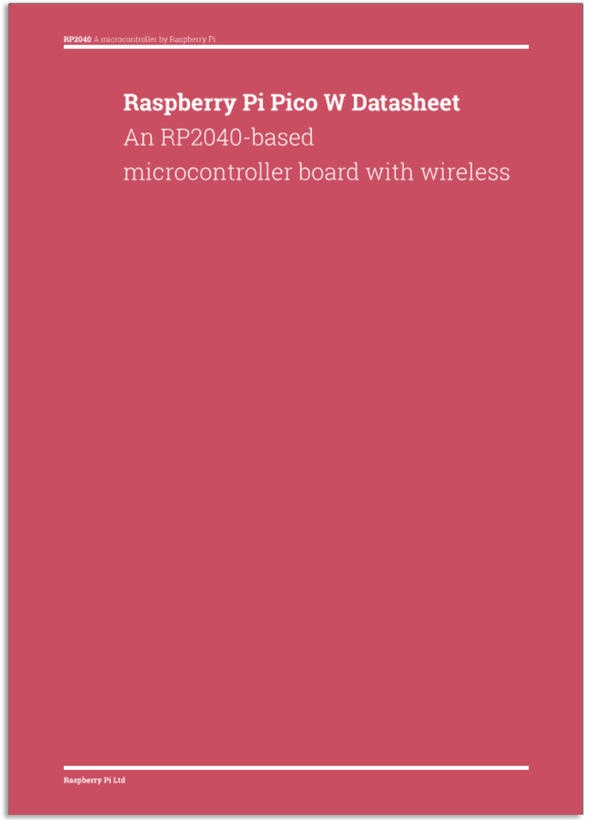
- Na placa de desenvolvimento Raspberry Pi Pico W, existe a viabilidade de configurar diferentes frequências de clock para conexão com periféricos internos e externos ao microcontrolador, tendo um limite de 133MHz.

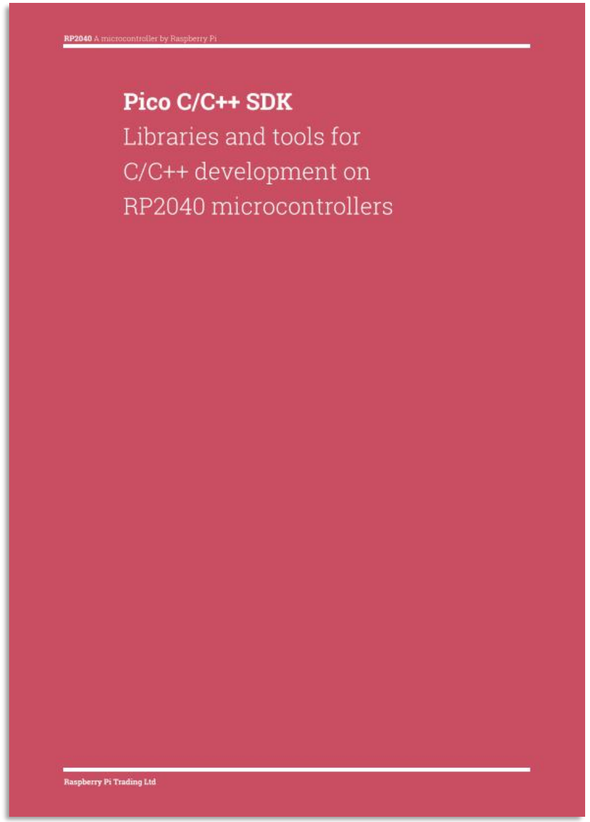
- Os temporizadores são ferramentas poderosas para contagem de eventos e controle de tempo, sendo muito úteis para os sistemas embarcados na implementação de rotinas periódicas e na definição de tempos de atuação precisos.

- Estes recursos podem contribuir na otimização da programação do RP2040, pois permitem a execução de tarefas em intervalos específicos sem a necessidade de intervenção constante do processador.

Bibliografia Recomendada







embarca tech | Residência Tecnológica em Sistemas Embarcados

Clock e Temporizadores

Unidade 4 | Capítulo 5 – Aula Síncrona (25/01/2025)

Executores: INSTITUTO FEDERAL Piauí · INSTITUTO FEDERAL Rio Grande do Norte · INSTITUTO FEDERAL Maranhão · INSTITUTO FEDERAL Ceará · cepedi · INSTITUTO HARDWARE BR

Coordenação: Softex

Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO · GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO